Cybersecurity

RESEARCH

Open Access

LP-HENN: fully homomorphic encryption accelerator with high energy efficiency

Zhuoyu Tian[1,2,3*], Lei Chen[4], Shengyu Fan[1,2,3], Xianglong Deng[1,2,3], Rui Hou[1,2,3], Dan Meng[1,2], Mingzhe Zhang[4*]

Abstract

Fully homomorphic encryption (FHE) enables direct computation on encrypted data without decryption, ensuring data privacy in cloud computing scenarios and preventing the leakage of sensitive information. However, the computational overhead of HE typically exceeds that of plaintext computation by 4 to 5 orders of magnitude, while energy consumption is 5 to 6 orders of magnitude higher. These substantial performance and energy overheads significantly hinder the widespread adoption of FHE. This paper proposed LP-HENN, a novel low-power and energy-efficient FHE accelerator architecture that leverages a RISC-V vector coprocessor and ReRAM crossbar arrays. LP-HENN targets power-constrained application scenarios such as edge devices, aiming to provide highly energy-efficient acceleration support for FHE applications. LP-HENN leverages the collaborative work of the vector processor and ReRAM crossbars, employing optimization strategies to achieve full pipelining and minimize memory access. Furthermore, this paper proposed a parameter selection model for early-stage architecture design, which achieves an optimal balance between performance and energy consumption through the collaborative optimization of multiple parameters. Experimental results show that, for an FHE-based convolutional neural network (HE-CNN) inference application, LP-HENN achieves a 31.82Ã- and 11920.56Ã- improvement in performance and energy efficiency, respectively, compared to CPU. Compared to FxHENN, the state-of-the-art FPGA-based FHE accelerator with high energy efficiency for edge devices, LP-HENN achieves a 2.36Ã- and 10.04Ã- improvement in performance and energy efficiency, respectively. The energy efficiency of LP-HENN is comparable to that of F1, the state-of-the-art ASIC FHE accelerator, while featuring a low power design suitable for edge computing.

Keywords Fully homomorphic encryption, Convolutional neural network, RISC-V, Resistive random access memory, Domain-specific architecture

Introduction

Fully Homomorphic Encryption (FHE) is a special type of encryption scheme that enables direct computations on encrypted data without requiring decryption. The encrypted result of these computations, when decrypted, yields an output identical to that obtained from performing the same computations on the original plaintext data. This capability facilitates the development of applications with end-to-end encrypted data processing, ensuring data security even in untrusted environments, as Fig. 1" illustrates.

Despite the tremendous potential of FHE, its significant overhead makes it challenging to be widely adopted in the real world. Currently, even with state-of-the-art FHE libraries (Chen et al. 2017; Al Badawi et al. 2022; Mouchet et al. 2020), homomorphic encryption computations on CPUs are 4-5 orders of magnitude slower than their plaintext counterparts. This enormous computational overhead not only reduces

*Correspondence:
Zhuoyu Tian
tianzhuoyu@iie.ac.cn
Mingzhe Zhang
mingzhe-zhang@outlook.com
[1] Institute of Information Engineering, CAS, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3] State Key Laboratory of Cyberspace Security Defense, Beijing, China
[4] Ant Group, Beijing, China

Springer Open

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

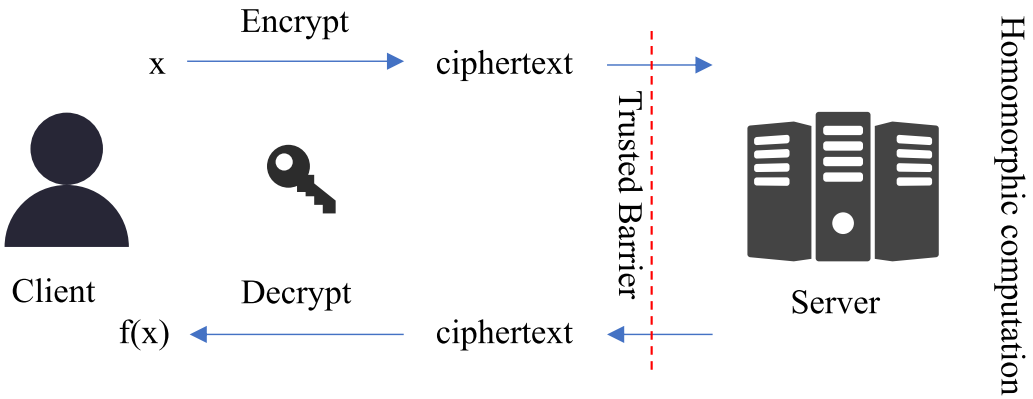


Fig. 1 FHE enables secure computation offloading to the cloud while preserving data privacy (Samardzic et al. 2021)

FHE performance but also significantly increases energy consumption. For instance, in neural network inference, using FHE on CPUs or GPUs results in 5 to 6 orders of magnitude increase in energy consumption compared to inference on plaintext data (Zhu et al. 2023). This substantial energy overhead implies prohibitively high costs for FHE applications. Therefore, mitigating the performance and power consumption overheads is crucial for promoting the widespread adoption of FHE.

To reduce the performance and energy consumption overheads of FHE, researchers have conducted extensive studies and proposed a series of FHE hardware accelerators (Samardzic et al. 2021; Samardzic et al. 2022; Kim et al. 2022a; b; Agrawal et al. 2023; Yang et al. 2023; Kim et al. 2023; Zhu et al. 2023). In the domain of edge computing, FxHENN (Zhu et al. 2023) is the first FPGA FHE accelerator supporting comprehensive end-to-end neural network applications. Compared to CPUs, FxHENN achieves a speedup of 13.49Ã- and an energy efficiency improvement of 1187.12Ã-.

FHE operations process long vectors (typically ranging from 16384 to 65536 elements in length), with the most of computations being element-wise operations on these vectors, thus exhibiting a high degree of data-level parallelism (DLP). Futhermore, FHE programs do not involve any branches, making them highly suitable for processing with vector processors. Vector processors are characterized by their superior energy efficiency while providing ample computational capability for applications with high DLP. Given the deterministic nature of the computation process, computation and memory access can be effectively overlapped with an appropriate scheduling strategy.

The extensive use of Number Theoretic Transform (NTT) in FHE introduces significant computational and memory access overhead, making it the primary performance bottleneck. Therefore, to improve the performance of FHE, the incorporation of additional modules dedicated to accelerating NTT computations is necessary. Prior research has demonstrated that Number Theoretic Transform (NTT) computations can be efficiently expressed using matrix operations (Fan et al. 2023). Resistive Random Access Memory (ReRAM) crossbar arrays excel at matrix–vector multiplication with high energy efficiency (Shafiee et al. 2016; Liu et al. 2022), as they can perform hundreds of parallel vector dot products. Furthermore, previous studies (Park et al. 2022) have demonstrated that ReRAM-based NTT computations not only achieve superior energy efficiency (12.7% lower energy consumption) but also significantly higher throughput (63.8Ã-) compared to ASIC implementations. Thus, leveraging ReRAM crossbar arrays for NTT acceleration emerges as a promising approach.

Based on these observations, this paper proposed LP-HENN, a highly energy-efficient architecture for FHE in edge computing scenarios. LP-HENN integrates a RISC-V vector coprocessor and ReRAM crossbar arrays to achieve high energy efficiency. The design of FHE accelerators involves numerous software and hardware parameters. The values of these parameters and their combinations profoundly influence the performance and energy efficiency of FHE accelerators. For example, unneccesary large parameters can unnecessari increase computational overhead and cause bad performance and high energy consumption. To choose the optimal software and hardware parameters in the early architectural design stage, we provide a parameter selection model.

Experimental results show that for the LoLa-CIFAR (Brutzkus et al. 2019) HE-CNN application, LP-HENN achieves a speedup of 31.82Ã- and an energy efficiency improvement of 11920.56Ã- compared to CPUs. Compared to FxHENN, LP-HENN achieves a speedup of 2.36Ã- and an energy efficiency improvement of 10.04Ã-.

Our contributions can be summarized as follows:

- We present LP-HENN, a novel architecture that combines a RISC-V vector coprocessor with ReRAM crossbar arrays, achieving high energy efficiency comparable to F1(Samardzic et al. 2021), the current state-of-the-art ASIC FHE accelerator, while featuring a low power design suitable for edge computing.
- We develop a parameter selection model for LP-HENN to choose the optimal software and hardware parameters during the early stage of architectural design to balance the accelerator's performance and energy consumption.
- We evaluate the performance and power consumption of LP-HENN using the LoLa (Brutzkus et al. 2019) neural networks. The experimental results demonstrate significant improvements in both performance and energy efficiency compared to CPUs and FxHENN, underscoring the efficacy of our proposed architecture.

Background

The LP-HENN architecture is applicable to FHE schemes such as BGV (Brakerski et al. 2014), BFV (Fan and Vercauteren 2012), and CKKS (Cheon et al. 2017), as their computations are similar. This paper focuses on CKKS over other popular homomorphic encryption schemes (BFV, BGV, and TFHE) for the following reasons:

- The CKKS scheme efficiently supports the encryption of fixed-point complex vectors and currently stands as the most efficient scheme for performing approximate homomorphic computations over real and complex numbers (Kim et al. 2022). This makes it particularly suitable for machine learning (ML) applications. In contrast, BFV and BGV schemes are designed for exact computations with integer data types, while TFHE operates on bits. Applying these alternative schemes to ML workloads requires quantization, and they lack efficient mechanisms for rounding real numbers, limiting their practicality for ML applications.
- While BFV and BGV support SIMD-style ciphertext packing (allowing multiple numbers in one ciphertext), their packing efficiency is substantially lower than CKKS (Badawi and Polyakov 2023). Due to their message encoding methodology, BFV and BGV restrict SIMD-encoded vectors to sizes of 2K or less, whereas CKKS supports vector sizes ranging from 32K to 64K. Consequently, BGV and BFV computations achieve significantly lower throughput compared to CKKS.

Section "CKKS scheme" introduces the basic concepts of the CKKS scheme, including the encryption and decryption process, as well as the homomorphic primitive operations supported by the CKKS scheme. Section "Common algorithm optimizations used by FHEaccelerators" presents common optimization techniques for CKKS, such as NTT, RNS-CKKS, and the generalized key-switching algorithm. Section "Hierarchical description of RNS-CKKS" provides a hierarchical description of the RNS-CKKS scheme. Section "Matrix-vector multiplication accelerator based on ReRAMcrossbars" describes how the matrix–vector multiplication accelerator based on ReRAM crossbars operates. The notation used throughout this paper is shown in Table 1.

CKKS scheme

The CKKS scheme(Cheon et al. 2017) has emerged as one of the most widely adopted FHE schemes. In this scheme, the message is an unencrypted complex vector $\boldsymbol{m}$ of length n. Before encryption, the message is encoded and transformed into a polynomial $\mathrm{P}_{\boldsymbol{m}}$ of degree $N-1$ within the polynomial ring $\mathcal{R}_Q$, where N is a power of 2 and satisfies $N/2 \geq n$. The encoding procedure applies an Inverse Discrete Fourier Transform (IDFT) to $\boldsymbol{m}$, followed by multiplication with a scaling factor Δ and subsequent rounding to integers:

$$\mathrm{P}_{\boldsymbol{m}} \simeq \Delta \cdot \mathrm{IDFT}(\boldsymbol{m}).$$

Subsequently, the encoded result is encrypted using the public key $(\mathrm{B}, \mathrm{A}) = (-\mathrm{A} \cdot \mathrm{S} + \mathrm{E}_{\mathrm{pk}}, \mathrm{A}) \in \mathcal{R}_Q^2$:

$$[\![\boldsymbol{m}]\!] = (\mathrm{B}_{\boldsymbol{m}}, \mathrm{A}_{\boldsymbol{m}}) = (\mathrm{B}, \mathrm{A}) + (\mathrm{P}_{\mathrm{m}} + \mathrm{E}_0, \mathrm{E}_1) \in \mathcal{R}_Q^2,$$

where S is the secret key polynomial, and E_{pk}, E_0, and E_1 are randomly sampled error polynomials, with $\mathrm{S}, \mathrm{E}_{\mathrm{pk}}, \mathrm{E}_0, \mathrm{E}_1 \in \mathcal{R}_Q$.

Decryption is the inverse of the encryption procedure. Initially, the polynomial $\mathrm{P}_{\boldsymbol{m}}$ is recovered through $\mathrm{B}_{\boldsymbol{m}} - \mathrm{A}_{\boldsymbol{m}} \cdot S = \mathrm{P}_{\boldsymbol{m}} + \mathrm{E}$, where the error term E in the result introduces an approximation error. Subsequently, the recovered polynomial is subjected to a Discrete Fourier Transform (DFT). The decryption process can be described as

$$\Delta^{-1} \cdot \mathrm{DFT}\,(\mathrm{B}_{\boldsymbol{m}} - \mathrm{A}_{\boldsymbol{m}} \cdot \mathrm{S}) = \Delta^{-1} \cdot \mathrm{DFT}\,(\mathrm{P}_{\boldsymbol{m}} + \mathrm{E}) \simeq \boldsymbol{m}.$$

The most important property of homomorphic encryption is its support for the FHE primitive operations, satisfying $\mathrm{Decrypt}(f([[\boldsymbol{m}]])) \approx f(\boldsymbol{m})$, which allows direct computation on ciphertexts. Table 2 illustrates the input and output of CKKS primitive operation primitive, along with their computations. The automorphism operation, denoted by ψ_r in Table 2, takes the i-th coefficient of the input polynomial as the $i \cdot 5^r \bmod N$-th coefficient of the output polynomial. After HMult and PMult, Rescale needs to be called to adjust the scaling factor of the resulting ciphertext from Δ^2 to Δ. Rescale effectively divides both the scaling factor and modulus of the ciphertext by Δ.

HMult and HRotate involve the `KeySwitch` operation, which converts a ciphertext under one secret key to a ciphertext under another secret key. The `KeySwitch` operation is defined as $\texttt{KeySwitch}(\mathrm{P}_{\boldsymbol{m}}, \mathbf{evk}) = P^{-1} \cdot \mathrm{P}_{\boldsymbol{m}} \cdot \mathbf{evk}$, where P is a large number satisfying $P \geq Q$. **evk** is the switching key, and $\mathbf{evk} \in \mathcal{R}_{PQ}^2$. The switching key used in HMult is $\mathbf{evk}_{\mathrm{mult}}$. For HRotate, each distinct rotation amount r corresponds to a unique switching key $\mathbf{evk}_{\mathrm{rot}}^{(r)}$.

Table 1 Notations and CKKS parameters

Notation	Description
$\boldsymbol{A}$	A bold uppercase letter represents a matrix.
$\boldsymbol{a}$	A bold lowercase letter represents a vector.
A	A non-bold upright uppercase letters represent a polynomial.
$[a]_q$	a modulo q
$x^{(i)}$	i-th bit of integer x
$\boldsymbol{A}_j$	j-th row of $\boldsymbol{A}_j$
$\mathbb{Z}_q^N$	Superscript N on a set denotes a vector of length N, the elements of which belong to the set.
$\boldsymbol{m}$	Message, a vector of n complex (real) numbers
$\mathcal{R}_Q$	Polynomial ring $\mathbb{Z}_Q/(X^N+1)$
$\mathrm{P}_{\boldsymbol{m}}$	Polynomial plaintext obtained by encoding $\boldsymbol{m}$
ct	A Ciphertext
$[[\boldsymbol{m}]]$	Ciphertext obtained by encoding, which is a polynomial pair, $[[\boldsymbol{m}]] \in \mathcal{R}_Q^2$
Δ	Scaling factor multiplied to polynomial during encoding
N	Number of polynomial coefficients, i.e., number of terms
Q	Initial polynomial modulus of ciphertext
q_j	Small RNS prime composing Q, $Q = \prod_{j=1}^{L} q_j$
L	Number of q_j composing Q
B	Bit length of q_j
P	A large number to reduce noise during `KeySwitch`
p_i	Small RNS prime composing P, $P = \prod_{i=1}^{K} p_i$
K	Number of p_i composing P
ℓ	Current level of ciphertext which is the number of remaining q_j in the modulus, $1 \leq \ell \leq L$
$\psi_{N,q}$	Twiddling factors of NTT
evk	Evaluation key used in `KeySwitch`
$\odot$	Hadamard multiplication (i.e. element-wise multiplication)
`dnum`	Decomposition number used in generalized `KeySwitch`, $\mathrm{K} = \lceil \frac{L}{\mathtt{dnum}} \rceil$
α	$\alpha = \lceil \frac{\ell}{\mathtt{dnum}} \rceil$, used in generalized `KeySwitch`
β	$\beta = \lceil \frac{\ell}{\alpha} \rceil$, used in generalized `KeySwitch`
limb	One row of DCRT representation of a polynomial, which is a vector of length N under the same modulus q_j
I	Number of lanes in the vector processor
N_1, N_2	$N = N_1 \cdot N_2$, used in the decomposition method of NTT (Sect. "Using decomposition method to compute NTT")

Common algorithm optimizations used by FHE accelerators

(I) NTT

The Number-Theoretic Transform (NTT) (Pollard 1971) is used for fast computation of polynomial multiplications. NTT is a variant of the Discrete Fourier Transform (DFT) in $\mathbb{Z}_q$. The input and output of $\mathrm{NTT}_{N,q}$ belongs to $\mathbb{Z}_q^N$, where N and q satisfy $N|q-1$. Let g be a primitive root of q, i.e., $g^{q-1} \equiv 1 \pmod{q}$, and the twiddle factor $\psi_{N,q}$ of NTT is defined as $g^{(q-1)/N} \bmod q$.

Given two polynomials $a(X), b(X) \in \mathcal{R}_q$ of degree $N-1$, the following holds:

$$\mathrm{NTT}_{N,q}(a(X) \cdot b(X)) = (\mathrm{NTT}_{N,q}(\boldsymbol{a}) \odot \mathrm{NTT}_{N,q}(\boldsymbol{b})) \bmod q,$$

where $\boldsymbol{a}$ and $\boldsymbol{b}$ represent the coefficient vectors of polynomials $a(X)$ and $b(X)$, respectively.

Therefore, the multiplication of two polynomials can be transformed into two NTTs, one element-wise multiplication, and one inverse NTT (INTT). When using the Cooley-Tukey algorithm, the time complexity of (I) NTT is $O(N \log N)$, and the overall time complexity of polynomial multiplication is $O(N \log N)$.

RNS-CKKS

The coefficients of $\mathrm{P}_{\boldsymbol{m}}$ are in $\mathbb{Z}_Q$, where Q is typically several hundred to thousands of bits. To circumvent the need for multi-precision arithmetic, RNS-CKKS(Cheon et al. 2019) employs the double Chinese remainder theorem (DCRT) representation of polynomials. Figure 2 illustrates the DCRT representation of a polynomial in $\mathcal{R}_Q$. The modulus Q is decomposed into L small primes $q_1, \ldots, q_L$, such that $Q = \prod_{j=1}^{L} q_j$. The RNS decomposition ($c \bmod q_1, \ldots, c \bmod q_L$) of any $c \in \mathbb{Z}_Q$ can be reconstructed to c by using the Chinese Remainder Theorem (CRT). By performing RNS decomposition on each coefficient of $\mathrm{P}_{\boldsymbol{m}}$, the polynomial is decomposed into L polynomials, and subsequently, the coefficient vector of each polynomial is transformed to the NTT domain, yielding an $L \times N$ matrix which is the DCRT representation of the polynomial. Each row of the DCRT matrix is referred to as a limb.

Under the DCRT representation, polynomial multiplication is element-wise multiplication of the DCRT matrices of the two polynomials, reducing the complexity from $O(N \log N)$ to $O(N)$. Given that NTT is a linear transformation, the computation of addition or automorphism does not require the conversion of the polynomial from DCRT representation to coefficient representation, and the computational complexity remains unchanged.

The Rescale operation, described in Sect. "CKKS scheme", divides the modulus of the ciphertext by Δ. In RNS-CKKS, all small primes q_j that comprise Q are selected to be approximately equal to Δ. The Rescale operation drops the last prime of the modulus, changing it from $\prod_{i=1}^{L} q_i$ to $\prod_{i=1}^{L-1} q_i$. The level ℓ represents the remaining number of primes in the ciphertext's modulus, which is initially L and decreases by 1 after each Rescale operation. Consequently, the level ℓ of the ciphertext decreases by 1 after both HMult and PMult operations.

To compute `KeySwitch`, RNS-CKKS introduces the `ModUp` and `ModDown` operations. `ModUp` converts the modulus of a polynomial from Q' to PQ' while keeping the polynomial approximately unchanged. Conversely, `ModDown` converts the modulus of a polynomial from PQ' to Q', and the polynomial coefficients are approximately divided by P.

The core computation in `ModUp` and `ModDown` is the fast basis conversion (`BConv`), which converts the RNS representation of a number from one basis to another. Let $\mathcal{C} = \{q_1, \ldots, q_\ell\}$ and $\mathcal{B} = \{p_1, \ldots, p_k\}$ be two basis, and $a \in \mathbb{Z}_{\prod_{j=1}^{\ell} q_j}$. The input of `BConv` is the RNS representation of a under the basis $\mathcal{C}$, denoted as $[a]_{\mathcal{C}} = ([a]_{q_1}, [a]_{q_2}, \ldots, [a]_{q_\ell}) \in \mathbb{Z}_{q_1} \times \cdots \times \mathbb{Z}_{q_\ell}$. The function $\mathtt{BConv}_{\mathcal{C}\to\mathcal{B}}([a]_{\mathcal{C}})$ outputs the approximate RNS

Table 2 CKKS primitive operations and their computation

Operation	Input and Output	Computation
PAdd	$\mathrm{PAdd}\left(\llbracket \boldsymbol{m} \rrbracket, \mathrm{P}_{\boldsymbol{m}'}\right) = \llbracket \boldsymbol{m} + \boldsymbol{m}' \rrbracket$	$\mathrm{PAdd}\left(\llbracket \boldsymbol{m} \rrbracket, \mathrm{P}_{\boldsymbol{m}'}\right) = (\mathrm{B}_{\boldsymbol{m}} + \mathrm{P}_{\boldsymbol{m}'}, \mathrm{A}_{\boldsymbol{m}})$
PMult	$\mathrm{PMult}\left(\llbracket \boldsymbol{m} \rrbracket, \mathrm{P}_{\boldsymbol{m}'}\right) = \llbracket \boldsymbol{m} \odot \boldsymbol{m}' \rrbracket$	$\mathrm{PMult}\left(\llbracket \boldsymbol{m} \rrbracket, \mathrm{P}_{\boldsymbol{m}'}\right) = (\mathrm{B}_{\boldsymbol{m}} \cdot \mathrm{P}_{\boldsymbol{m}'}, \mathrm{A}_{\boldsymbol{m}} \cdot \mathrm{P}_{\boldsymbol{m}'})$
HAdd	$\mathrm{HAdd}\left(\llbracket \boldsymbol{m} \rrbracket, \llbracket \boldsymbol{m}' \rrbracket\right) = \llbracket \boldsymbol{m} + \boldsymbol{m}' \rrbracket$	$\mathrm{HAdd}\left(\llbracket \boldsymbol{m} \rrbracket, \llbracket \boldsymbol{m}' \rrbracket\right) = (\mathrm{B}_{\boldsymbol{m}} + \mathrm{B}_{\boldsymbol{m}'}, \mathrm{A}_{\boldsymbol{m}} + \mathrm{A}_{\boldsymbol{m}'})$
HMult	$\mathrm{HMult}\left(\llbracket \boldsymbol{m} \rrbracket, \llbracket \boldsymbol{m}' \rrbracket\right) = \llbracket \boldsymbol{m} \odot \boldsymbol{m}' \rrbracket$	$\mathrm{HMult}\left(\llbracket \boldsymbol{m} \rrbracket, \llbracket \boldsymbol{m}' \rrbracket\right) = \mathtt{KeySwitch}\left(\mathrm{A}_{\boldsymbol{m}} \cdot \mathrm{A}_{\boldsymbol{m}'}, \mathbf{evk}_{\mathrm{mult}}\right) + (\mathrm{B}_{\boldsymbol{m}} \cdot \mathrm{B}_{\boldsymbol{m}'}, \mathrm{A}_{\boldsymbol{m}} \cdot \mathrm{B}_{\boldsymbol{m}'} + \mathrm{A}_{\boldsymbol{m}'} \cdot \mathrm{B}_{\boldsymbol{m}})$
HRotate	$\mathrm{HRot}\left(\llbracket \boldsymbol{m} \rrbracket, r\right) = \llbracket \boldsymbol{m} \ll r \rrbracket$	$\mathrm{HRot}\left(\llbracket \boldsymbol{m} \rrbracket, r\right) = \mathtt{KeySwitch}\left(\psi_r(\mathrm{A}_{\boldsymbol{m}}), \mathbf{evk}_{\mathrm{rot}}^{(r)}\right) + (\psi_r(\mathrm{B}_{\boldsymbol{m}}), \mathbf{0})$
Rescale	$\mathrm{Rescale}\left(\llbracket \boldsymbol{m} \rrbracket\right) = \llbracket \Delta^{-1} \cdot \boldsymbol{m} \rrbracket$	$\mathrm{Rescale}\left(\llbracket \boldsymbol{m} \rrbracket\right) = \left(\Delta^{-1} \cdot \mathrm{B}_{\boldsymbol{m}}, \Delta^{-1} \cdot \mathrm{A}_{\boldsymbol{m}}\right)$

representation of a under the basis $\mathcal{B}$, denoted as $[a]_{\mathcal{B}} \in \mathbb{Z}_{p_1} \times \cdots \times \mathbb{Z}_{p_k}$. The computation of $\mathrm{BConv}_{\mathcal{C}\to\mathcal{B}}([a]_{\mathcal{C}})$ is given by:

$$\mathtt{BConv}_{\mathcal{C}\to\mathcal{B}}([a]_{\mathcal{C}}) = \left(\left(\sum_{j=1}^{\ell}\left[[a]_{q_j}\cdot\hat{q}_j^{-1}\right]_{q_j}\cdot[\hat{q}_j]_{p_i}\right)(\mathrm{mod}\, p_i)\right)_{1\le i\le k},$$

where $\hat{q}_j = \prod_{i=1\wedge i\neq j}^{\ell} q_i$.

Generalized key-switching algorithm

The generalized key-switching algorithm (also referred to as the hybrid key-switching algorithm) (Han and Ki 2020) can improve the performance of `KeySwitch`, a time-consuming operation that dominates the runtime in FHE workloads. This algorithm introduces a new parameter `dnum`, which extends the switching key **evk** from a single polynomial pair to `dnum` polynomial pairs. The generalized key-switching algorithm first decomposes the input polynomial ($\ell \times N$ matrix) into $\beta = \lceil \frac{\ell}{\alpha} \rceil$ groups (`DComp`), where each group comprises $\alpha = \lceil L/\mathtt{dnum} \rceil$ moduli from $(q_1, q_2, \ldots, q_\ell)$ and has a shape of $\alpha \times N$. Subsequently, `ModUp` is performed on each group, yielding an output shape of $(\alpha\beta + k) \times N$. These β outputs are then multiplied element-wise with the first β polynomial pairs of **evk** and summed up, a process referred to as `InnerProduct`. The output is an $(\alpha\beta + k) \times N$ matrix. Finally, the `ModDown` operation is applied to the summation result, generating an output shape of $\ell \times N$.

The choice of `dnum` has a significant impact on the performance of KeySwitch. Experiments (Han and Ki 2020) demonstrate that the optimal `dnum` can result in a 2Ã-performance improvement compared to the worst case.

Hierarchical description of RNS-CKKS

From a computational perspective, an FHE application built on the CKKS scheme can be divided into the following four layers:

- Application layer: Applications constructed upon the CKKS scheme, such as convolutional neural networks (CNNs) (Lee et al. 2022) and logistic regression (Han et al. 2019).
- FHE primitive layer: FHE primitive operations described in Sect. "CKKS scheme", including `HAdd`, `HMult`, `HRotate`, `PMult`, `PAdd`, and `Rescale`.
- FHE Operator layer: The operators that compose the primitives operations, including `ModUp`, `ModDown`, `KeySwitch`, `BConv`, `DComp`, and `InnerProduct`.
- Vector operation layer: The various FHE operators are composed of the following basic vector operations: vector modular addition (`VecAddMod`), vector modular subtraction (`VecSubMod`), vector modular multiplication (`VecMulMod`), vector modulo (`VecMod`), vector addition (`VecAdd`), vector multiplication (`VecMul`), automorphism, and (I)NTT.

The composition of the RNS-CKKS scheme's primitives, FHE operators is summarized in Table 3.

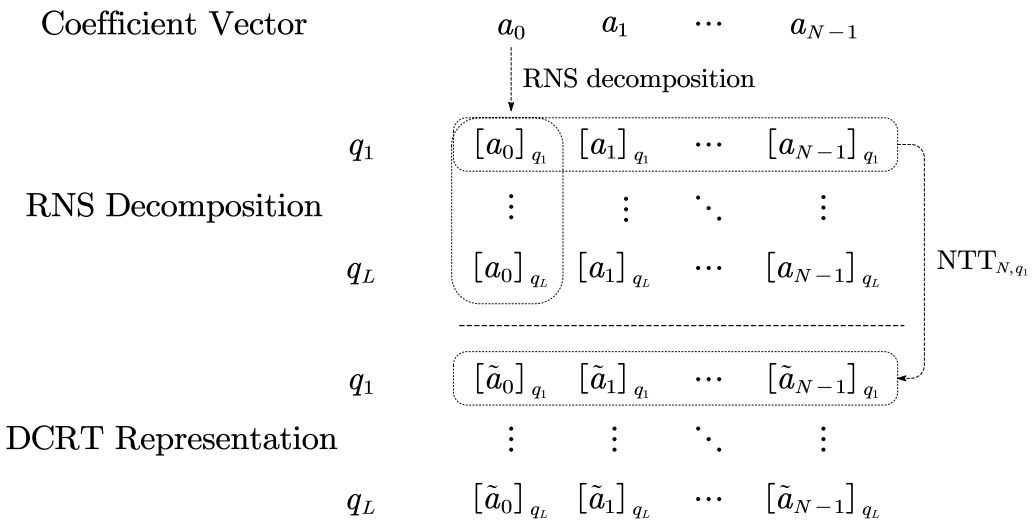


Fig. 2 DCRT representation of a polynomial

Matrix–vector multiplication accelerator based on ReRAM crossbars

The matrix–vector multiplication accelerator based on ReRAM crossbars is illustrated in Fig. 3. The accelerator comprises ReRAM crossbars, digital-to-analog converters (DACs), analog-to-digital converters (ADCs), sample-and-hold (S&H) circuits, shifters, and adders, which work in concert to facilitate efficient matrix–vector multiplication.

ReRAM crossbars perform matrix–vector multiplication in the analog domain. The input vector is converted into voltages $V_0, V_1, \ldots, V_{m-1}$ via DACs to drive the cells on the corresponding wordlines. Let the conductances of the cells in a single column of the crossbar be $G_0, G_1, \ldots, G_{m-1}$. According to Kirchhoff's law, the output current on the bitline is $\sum_{i=0}^{m-1} V_i G_i$, thus completing the dot product operation between the input vector $(V_0, V_1, \ldots, V_{m-1})$ and the conductance vector $(G_0, G_1, \ldots, G_{m-1})$. Subsequently, the S&H circuit samples the bitline current and forwards it to the ADC to convert the analog current signal into a digital signal. As a number may be distributed across multiple cells, the output result of each bitline corresponds to a partial sum. These partial sums are then accumulated using shifters and adders to obtain the final dot product result. The columns of the crossbar array operate in parallel, enabling an mÃ-n crossbar array to simultaneously compute n dot products of vectors with length m, thereby achieving a high degree of parallelism. This architecture serves as a low-power, high-efficiency dot product engine, making it well-suited for matrix–vector multiplication tasks.

Practical applications and significance

This research on low-power homomorphic encryption accelerators for edge computing presents significant practical applications.

Scenario 1: Edge computing-based facial recognition systems with homomorphic encryption. While facial recognition technology has been widely deployed, particularly in security systems for suspect detection, traditional face recognition systems face two major challenges. First, processing large crowds creates substantial computational and storage overhead for central servers. Offloading these computations to edge devices can significantly reduce the data center burden. Second, protecting the security of feature vector databases is crucial, as directly storing facial feature vectors can compromise both user privacy and system security (Naresh Boddeti 2018). Edge homomorphic accelerators effectively address both challenges by storing encrypted feature vector databases on edge devices and computing Euclidean distances (or similar matching metrics) between plaintext input data and encrypted database vectors. The encrypted results are then transmitted to central servers for decryption and match determination. This approach distributes computational and storage loads while ensuring database security through encryption. However, homomorphic encryption operations are computationally intensive, leading to high latency on conventional hardware. Therefore, specialized accelerators are necessary to achieve acceptable latency levels.

Scenario 2: Protection of sensitive biometric data such as fingerprints. Edge homomorphic encryption accelerators enable effective protection of sensitive biometric information through encrypted processing. After

Table 3 Decomposition of Primitive Operations and Operators in the RNS-CKKS Scheme

Primitive Operation/Operator	Constituent Vector Operations and Operators
`HMult`	`KeySwitch`, `VecAddMod`, `VecMulMod`
`HAdd`	`VecAddMod`
`HMult`	`VecMulMod`, `VecAddMod`
`PMult`	`VecMulMod`
`PAdd`	`VecAddMod`
`HRotate`	`Automorphism`, `KeySwitch`, `VecAddMod`
`Rescale`	(I)NTT, `VecSubMod`, `VecMulMod`
`KeySwitch`	`DComp`, `ModUp`, `InnerProduct`, `ModDown`
`DComp`	`VecMulMod`
`ModUp`	(I)NTT, `BConv`
`ModDown`	(I)NTT, `BConv`, `VecSubMod`, `VecMulMod`
`BConv`	`VecMulMod`, `VecAdd`, `VecMul`, `VecMod`
`InnerProduct`	`VecAdd`, `VecMul`, `VecMod`

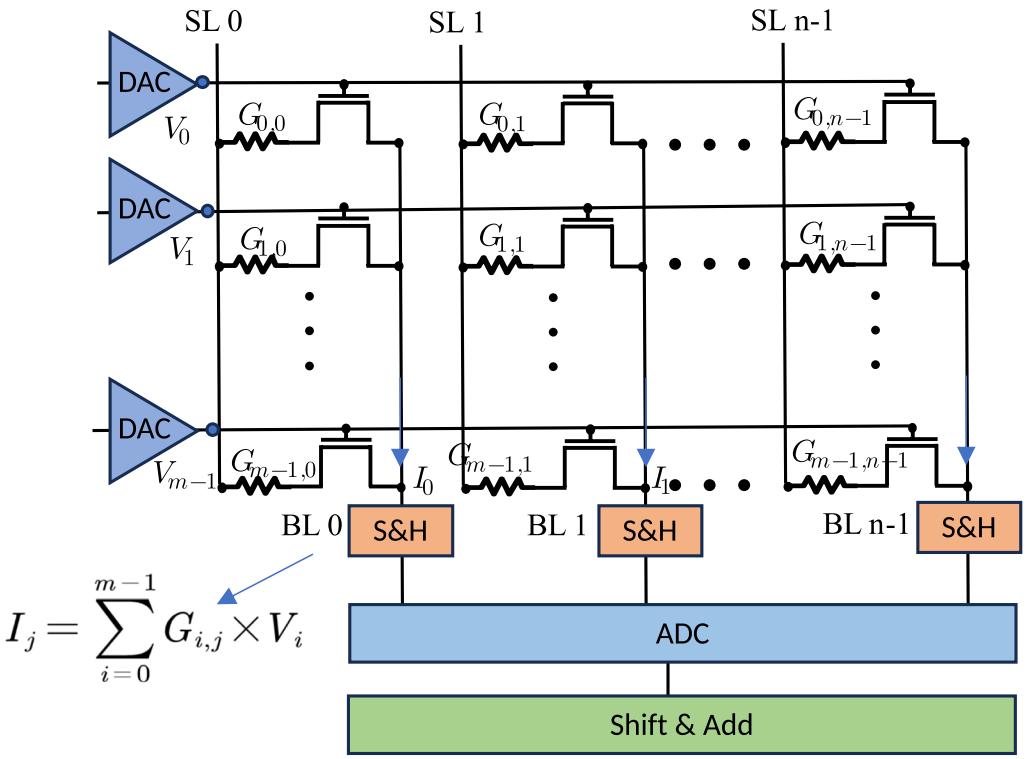


Fig. 3 Architecture and operation of the ReRAM crossbar-based matrix–vector multiplication accelerator

collecting user fingerprints, edge devices encrypt the data and perform classification using Convolutional Neural Networks (CNNs) in the encrypted domain (with model weights stored in either plaintext or ciphertext, depending on whether weight protection is required). This approach offers multiple advantages: (1) fingerprint data remains encrypted throughout processing, protecting against device compromise, (2) edge-based computation reduces data transmission and latency, and (3) the solution flexibly accommodates various security requirements-encrypting weights when the model is sensitive or using plaintext weights for improved efficiency when the model is public. However, the significant computational overhead of HE-CNN operations presents efficiency challenges, necessitating dedicated homomorphic encryption accelerators to maintain practical latency levels. This technology extends beyond fingerprint recognition to other biometric applications, including iris and voice recognition, demonstrating broad potential.

Energy efficiency is a critical design consideration in edge computing scenarios. First, many edge devices rely on battery power or have limited power supply conditions, making power consumption directly impact device operation time and user experience. Second, edge devices often operate in complex environments with constrained cooling capabilities. Excessive power consumption can lead to temperature increases, affecting system stability and potentially triggering thermal throttling, particularly in computation-intensive applications like homomorphic encryption. Moreover, given the massive scale of edge device deployment, even small per-device power reductions of a few watts can achieve significant energy savings across the entire device population, contributing substantially to reduced energy consumption and operational costs.

Related work

High-end FHE accelerators CPU-based FHE libraries, such as SEAL(Chen et al. 2017), OpenFHE(Al Badawi et al. 2022), and Lattigo(Mouchet et al. 2020), struggle to meet the performance requirements of practical FHE applications. GPU accelerators have the advantage of readily exploiting the well-established cloud ecosystem built around GPUs. GPU accelerators can achieve speedups ranging from tens to nearly a hundred times compared to CPUs, such as 100x(Jung et al. 2021) based on the NVIDIA Tesla V100 and TensorFHE(Fan et al. 2023) based on the NVIDIA A100. However, the performance

of GPUs is constrained by the limited on-chip buffer size. GME(Shivdikar et al. 2023), based on the AMD CDNA GPU architecture, mitigates the limited on-chip size by integrating a lightweight on-chip compute unit (CU)-side hierarchical interconnect and incorporating a series of microarchitecture improvements, resulting in performance enhancements of hundreds of times compared to CPUs. FPGA accelerators (Agrawal et al. 2023; Yang et al. 2023) can achieve performance close to that of GPUs while consuming less power, with the computational cost being approximately one-third of that of GPUs for the same workload (Turan et al. 2020). However, FPGAs are limited by their operating frequency and resources, which restricts their acceleration potential. Currently, the best-performing FHE accelerators are ASIC accelerators (Kim et al. 2023, 2022b, a; Samardzic et al. 2022, 2021), which can achieve thousands of times performance improvement compared to CPUs. These accelerators primarily target high-performance ciphertext computation needs in cloud computing scenarios rather than edge computing scenarios.

Low-power accelerators for edge devices FHE accelerators designed for edge computing scenarios(Zhu et al. 2023; Turan et al. 2020) are primarily based on FPGAs due to their high energy efficiency. HEAWS (Turan et al. 2020), an accelerator for the FV scheme based on AWS FPGAs, can only support low-level operations and cannot run deeper neural networks. FxHEEN (Zhu et al. 2023), the first full-fledged FPGA acceleration framework for HE-CNN inference based on the CKKS scheme, achieves a performance ten times that of CPUs and an energy efficiency thousands of times higher. In comparison, the LP-HENN architecture offers even higher energy efficiency and can support larger-scale applications.

Application of RISC-V processors in FHE acceleration To the best of our knowledge, LP-HENN is the first to utilize a RISC-V vector processor for FHE acceleration. Rise(Azad et al. 2023) and Race(Azad et al. 2022) employ RISC-V to accelerate client-side computation, which includes encryption and decryption, rather than ciphertext computation.

Processing in-Memory (PIM) in FHE acceleration The majority of PIM research in FHE acceleration concentrates on the acceleration of NTT, as it is the primary bottleneck in FHE workloads. NTT-PIM (Park et al. 2023) accelerates NTT based on DRAM, BP-NTT (Zhang et al. 2023) accelerates NTT based on SRAM, and CryptoPIM (Nejatollahi et al. 2020) and RM-NTT (Park et al. 2022) accelerate NTT based on ReRAM. However, their latencies are much larger than that of LP-HENN's NTTU, making it challenging to satisfy low-latency demands. Moreover, BP-NTT and RM-NTT only support small values of N (not more than 1024). FHE-PIM (Gupta and Rosing 2021) and MemFHE (Gupta et al. 2024) accelerate the complete FHE scheme rather than just NTT. FHE-PIM (Gupta and Rosing 2021) employs ReRAM to accelerate the FHEW scheme (Ducas and Micciancio 2015) by implementing a series of primitive operations, such as polynomial addition and multiplication, and adopts the NTT implementation from CryptoPIM. Compared to CPU, FHE-PIM achieves an estimated average throughput improvement of 88397Ã- for FHE arithmetic operations. MemFHE uses ReRAM to accelerate the RGSW scheme (Gentry et al. 2013) and demonstrates up to 20kÃ- (265Ã-) faster performance than CPU (GPU) for FHE arithmetic operations. However, these works accelerate FHE schemes based on GSW, rather than the CKKS scheme.

Motivation

To understand the challenges faced in the low-power FHE accelerators, this section presents an in-depth analysis of the characteristics of FHE applications from the perspectives of energy consumption, memory access, and parameter selection, based on the HE-CNN applications LoLa-MNIST and LoLa-CIFAR(Brutzkus et al. 2019). The results indicate that the NTT is the primary bottleneck, dominating both performance and energy consumption. Additionally, the selection of parameters in the CKKS scheme and the architectural design strongly influences the performance, energy efficiency, and resource utilization of FHE accelerators. The potential optimization opportunities derived from these findings are then summarized.

Energy and performance implications of NTT in FHE acceleration

Our first observation is that NTT operations constitute the primary bottleneck in terms of both energy consumption and computational performance, making their optimization crucial for advancing FHE accelerator designs.

From an energy consumption perspective, our analysis of (I)NTT operations in LoLa-CIFAR, based on Xeon processor energy consumption models, reveals significant findings. As shown in Fig. 4, NTT computation dominates the total energy consumption at 70.76%, while its associated memory access operations account for an additional 27.4%. The implementation utilizes the Cooley-Tukey butterfly algorithm for (I)NTT operations, with experimental configuration detailed in Table 4. Even in F1(Samardzic et al. 2021), a state-of-the-art ASIC implementation of FHE featuring highly optimized NTT units, these units still constitute 42.7% of the total energy consumption.

Table 4 Experimental configuration for NTT energy consumption analysis

Parameter	Value
CPU model	Intel Xeon E-2186G Processor
Power	95 W TDP
#Cores	6
#Hyperthreads	12
Energy per cache read/write	1 nJ
Cache line size	64 B

From a performance analysis perspective, Table 5 presents the proportion of computation (measured by the number of multiplications and additions) and memory access for NTT in LoLa-MNIST and LoLa-CIFAR. In both applications, NTT accounts for approximately 70% of the computation, while its memory access proportion exceeds 92%. Moreover, NTT serves as a critical step in FHE computations. NTT can only begin after all the previous results are obtained, and subsequent computations can only be initiated upon the completion of NTT. Consequently, NTT is the primary performance bottleneck in FHE applications. This finding aligns with previous research (Fan et al. 2023), which demonstrates that NTT operations dominate the overall runtime, accounting for more than 90% across all benchmarks. The suboptimal performance of NTT can be primarily attributed to the strict data dependencies between the stages of the Cooley-Tukey butterfly algorithm. Each stage must wait for the previous stage to complete its computation, write the results back to memory, and then read the results written by the previous stage before proceeding with its own computation. All $\log_2 N$ stages require all N elements to be read and written once, resulting in a significant amount of memory access. Furthermore, the strict data dependencies prevent any overlap between the computation time and memory access time in NTT, exacerbating the performance bottleneck.

Therefore, optimizing both performance and energy efficiency of NTT operations is crucial for enhancing FHE accelerator designs. ReRAM crossbar architecture presents a promising opportunity by enabling NTT computation through matrix–vector multiplication, offering high parallelism and significantly reduced data dependencies. Under the same manufacturing process node, this approach demonstrates 40% lower energy consumption compared to F1 (Park et al. 2022; Samardzic et al. 2021).

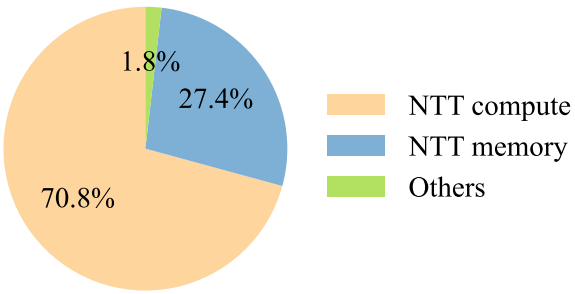


Fig. 4 Energy breakdown of LoLa-CIFAR evaluated using Xeon processor energy consumption models, with (I)NTT operations performed by the Cooley-Tukey butterfly algorithm

Parameter selection

The design of FHE accelerators involves numerous software and hardware parameters, which collectively constitute a large parameter space. The software parameters are the parameters of the CKKS scheme, including the number of polynomial terms (N), the initial level of ciphertexts (L), the decomposition number (`dnum`) in `KeySwitch`, and the bit length of the small primes (q_j) (B). The hardware parameters include the size of the on-chip cache, the capacity of the Dynamic Random-Access Memory (DRAM), and the number of functional units.

The values of these parameters and their combinations profoundly influence the performance and energy efficiency of FHE accelerators. The computational overhead of homomorphic operations is proportional to N. For example, when $N = 16384$, the computational cost of homomorphic operations is twice that of $N = 8192$. However, N also impacts the security level, and a small N may fail to satisfy the security requirements. Furthermore, N determines the vector length in FHE applications, and different values of N lead to a varying number of primitive operations. The computational overhead of primitive operations is proportional to ℓ, implying that a smaller L yields reduced computational

costs. Nevertheless, as described in Sect. "RNS-CKKS", homomorphic multiplications decrease the level. With a small L, there may not be sufficient multiplicative depth to complete the entire FHE workload. Regarding B, for devices with fixed word-sizes (32 or 64), smaller B ($B < 32$) exhibits superior computational efficiency compared to $B > 32$. Moreover, larger B values necessitate multi-precision operations, which suffer from suboptimal computational efficiency. However, an excessively small B affects the precision of FHE applications and may lead to incorrect results. The computational overhead does not exhibit a strictly linear relationship with `dnum`. Instead, there exists an optimal value of `dnum`, determined by other parameters, that minimizes the computational overhead of `KeySwitch`. The hardware parameters depend on the requirements of the specific application. Unnecessarily excessive hardware resources lead to low utilization and energy waste, while insufficient hardware resources result in poor performance. Consequently, selection of parameters has a multifaceted influence on performance and security level. Moreover, the optimal parameter choice is interdependent, making manual parameter selection a complex and demanding task.

Therefore, to efficiently search for the optimal parameter combination within the parameter space during the design phase, it is necessary to construct an automatic parameter selection model.

Opportunities

Based on the above observations, we identify the following opportunities to improve the energy efficiency of FHE accelerators:

- From the perspectives of latency, computational overhead, memory access, and power consumption, NTT is the primary bottleneck in FHE applications. Consequently, optimizing (I)NTT will have a significant impact on the performance and power consumption of FHE accelerators. Previous studies (Park et al. 2022) have demonstrated that ReRAM-based NTT computations not only achieve superior energy efficiency (12.7% lower energy consumption) but also significantly higher throughput (63.8Ã-) compared to ASIC implementations. Therefore, leveraging ReRAM crossbars to perform NTT may substantially improve both the performance and energy efficiency of FHE accelerators. Furthermore, The operators that constitute FHE applications can be converted into vector operations. Given the superior energy efficiency of vector processors, they present a compelling option for implementing most FHE operators.
- The selection of parameters in the CKKS scheme and the architectural design profoundly impacts the performance, energy efficiency, and resource utilization of FHE accelerators. However, the huge parameter search space and complex interdepencies between these parameters make manual parameter selection a challenging task. Consequently, an automatic parameter selection model is essential for design of FHE accelerators.

Based on the above analysis, we proposed the LP-HENN architecture and a parameter selection model to optimize energy efficiency. In the following sections, we will present the LP-HENN architecture and parameter selection model in detail.

LP-HENN architecture

Overview

Function units The architecture of LP-HENN, depicted in Fig. 5, comprises two primary computational components: a dedicated NTT unit (NTTU) composed of ReRAM crossbars and a vector unit, which is essentially a RISC-V vector processor Ara(Cavalcante et al. 2019; Perotti et al. 2022). As explained in Sect. "Hierarchical description of RNS-CKKS", most of the computations in FHE applications are vector operations. Therefore, LP-HENN employs a vector processor as the principal computation unit for these vector operations. Ara works as a coprocessor of CVA6, an open-source Linux-capable application-class core running at up to 1.7G Hz.

Table 5 Proportion of computation and memory access of (I)NTT in LoLa-MNIST and LoLa-CIFAR

	NTT computation	NTT memory access
LoLa-MNIST	70.6%	92.2%
LoLa-CIFAR	68.6%	93.3%

Section "Energy and performance implications of NTT in FHE acceleration" highlights that computing NTT using the Cooley-Tukey butterfly algorithm leads to a significant amount of memory access. Furthermore, the strict data dependency makes it impossible to overlap the computation time and memory access time of NTT, rendering NTT computation a performance and energy consumption bottleneck. In contrast, using matrix–vector multiplication to compute NTT exhibits a high degree of parallelism and low data dependency (Fan et al. 2023), equivalent to N independent dot products of length N. However, the computational complexity of this approach reaches $O(N^2)$, which results in an even larger computational workload and worse performance for vector processors.

To address this challenge, LP-HENN leverages the highly energy-efficient matrix–vector multiplication acceleration capabilities of ReRAM crossbars, as described in Sect. "Matrix-vector multiplication accelerator based on ReRAM crossbars", and offloads the NTT computations onto these crossbars for processing.

The vector unit and the NTT unit exchange data through the Scratchpad Memory (SPM), which is connected to the off-chip memory. This architectural design enables LP-HENN to efficiently compute non-NTT operations using the vector coprocessor while harnessing the power of ReRAM crossbars for accelerating NTT computations, ultimately optimizing the overall performance and energy efficiency of FHE applications.

Memory system The memory system of LP-HENN consists of a VRF, L1 cache, SPM, and DRAM. As described in Sect. ""Vector unit: structure and workflow", the VRF capacity is related to the number of lanes l. There are 32 vector registers, each with a length of VLEN $= 1024 * l$ bits. Ara's memory bandwidth is also related to the number of lanes (l), which is 2 B/DP-FLOP. Considering the case of $l = 4$, Ara's memory bandwidth is 16 GB/s, and the VRF capacity is 16 KB.

CVA6 has only one level of cache. The L1 D-cache in CVA6 has a capacity of 32 KB, with the cache line size matching the SPM port width. SPM essentially serves as a large cache, composed of a single-port bank with a port width of 1024 B and a capacity of 4587520 KB. The method for calculating the SPM capacity is provided in Sect. "SPM capacity". Due to the static characteristics of FHE programs, SPM employs a prefetching mechanism to read data from off-chip memory. By preloading ciphertexts, plaintexts, and evaluation keys for subsequent operations from DRAM, SPM effectively hides the main memory access latency. SPM operates at a frequency of 1 GHz, providing a bandwidth of 1 TB/s. The DRAM used in LP-HENN is DDR4 memory, which provides a bandwidth of 19.2 GB/s.

The chunk&fusion optimization discussed in Sect. "Chunk&fusion optimization" is crucial for enhancing the performance of LP-HENN, given the limited capacity of the VRF and cache.

RISC-V vector coprocessor

Vector unit: structure and workflow

For the vector processor, LP-HENN adopts Ara (Cavalcante et al. 2019), a 1 GHz 64-bit vector coprocessor

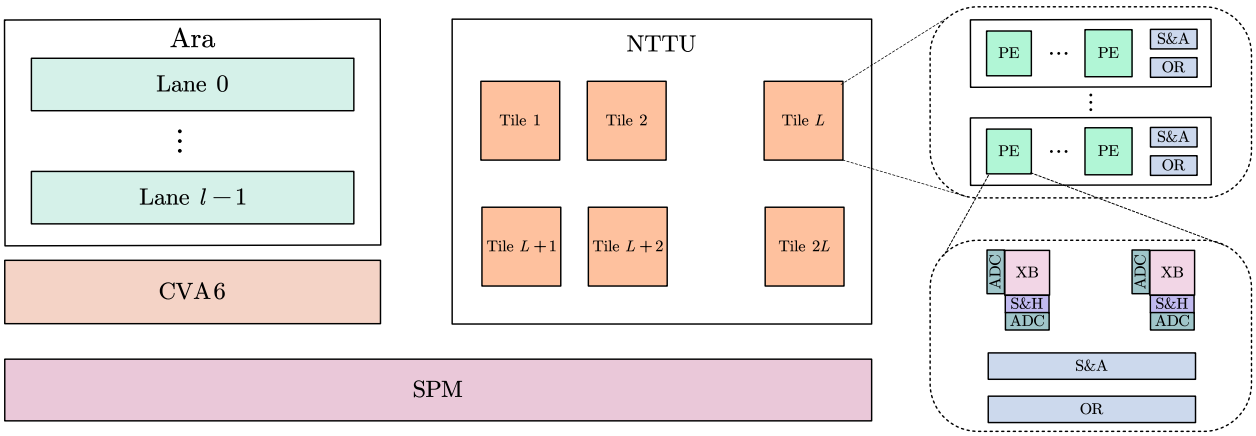


Fig. 5 Overview of the LP-HENN architecture. XB denotes crossbars; S&A represents shifter and adder units for accumulating results from multiple PEs and crossbars; OR indicates output registers for storing S&A results

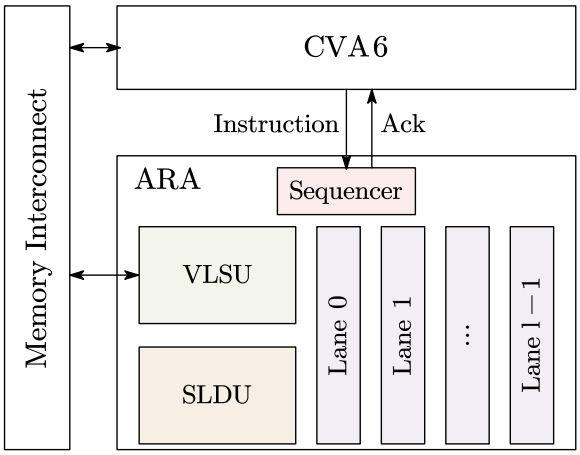


Fig. 6 Structure of the Ara RISC-V vector coprocessor

based on the RISC-V Vector Extension (RVV) 1.0 specification (RISC-V 2023). Ara achieves remarkable energy efficiency, reaching 41 DP-GFLOPS/W, outperforming comparable vector processors. The RVV specification defines two fixed parameters for RISC-V vector processors: ELEN and VLEN. ELEN represents the maximum size in bits of a vector element that any operation can produce or consume, while VLEN denotes the maximum number in bits of a single vector operand. Ara's ELEN is 64 bits, and its VLEN is determined by the number of lanes (l), where VLEN $= 1024 \times l$ bits.

Figure 6 illustrates the architecture of Ara, which collaborates with the CVA6 (Zaruba and Benini 2019), a RISC-V 64-bit scalar processor. Ara consists of a main sequencer, a slide unit (SLDU), a vector load/store unit (VLSU), and multiple lanes. The main sequencer monitors vector instructions running on Ara, dispatches vector instructions to execution units, and acknowledges with CVA6. The SLDU handles instructions that require access to the vector register files (VRF) across multiple lanes, such as vector slide instructions like `vslideup` and vector register gather instructions like `vrgather`. The VLSU performs data transfers via an AXI interface and supports unit-stride, constant-stride, and indexed load and store instructions. The addresses of unit-stride memory instructions are merged to reduce the number of memory requests. Lanes serve as the primary execution units, with each lane containing an Arithmetic Logic Unit (ALU), a Floating Point Unit (FPU), an integer multiplier (MUL), and a portion of the VRF. These three execution units have the same throughput of 64 bits/cycle, supporting 1 64-bit operation, 2 32-bit operations, 4 16-bit operations, or 8 8-bit operations per cycle. The VRF within each lane comprises eight single-ported banks with a width of 64 bits. Ara is scalable, supporting different configurations of the number of lanes l, which can be set to 2, 4, 8, or 16.

Ara makes modifications to CVA6. During the decoding stage, CVA6 dispatches vector instructions to Ara. Upon confirming that the instructions will not trigger exceptions, Ara sends an acknowledgment signal to CVA6 through the sequencer. Then Ara executes instructions sequentially without speculation. As FHE applications exhibit completely deterministic behavior and do not require branch instructions, sequential execution does not incur performance loss. Multiple lanes execute a vector instruction simultaneously. Since FHE operators do not involve instructions that require handling by the SLDU, and each lane has an independent VRF storing a portion of the vector operands, execution across lanes is relatively independent.

Basic vector operations and their implementations

According to Sect. "Hierarchical description of RNS-CKKS", FHE operations can ultimately be decomposed into element-wise vector operations, automorphism (Sect. "CKKS scheme"), and (I) NTT (Sect. "CKKS scheme"). The (I)NTT is collaboratively completed by ReRAM crossbars and the vector unit, and its computational process will be presented in Sect. "NTT Unit". The vector unit is responsible for performing the following vector operations:

`VecAddMod` Element-wise addition of two vectors followed by modulo q, $\texttt{VecAddMod}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = (\boldsymbol{\nu_1} + \boldsymbol{\nu_2}) \bmod q$.
`VecSubMod` Element-wise subtraction of two vectors followed by modulo q, $\texttt{VecSubMod}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = (\boldsymbol{\nu_1} - \boldsymbol{\nu_2}) \bmod q$.
`VecAdd` Element-wise addition of two vectors, $\texttt{VecAdd}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = \boldsymbol{\nu_1} + \boldsymbol{\nu_2}$.
`VecMulMontMod` Element-wise multiplication of two vectors followed by modulo q using the Montgomery reduction algorithm (Montgomery 1985), where one of the vectors is a known constant vector (a constraint of Montgomery reduction), $\texttt{VecMulMontMod}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = (\boldsymbol{\nu_1} \cdot \boldsymbol{\nu_2}) \bmod q$.
`VecMul` Element-wise multiplication of two vectors, $\texttt{VecMul}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = \boldsymbol{\nu_1} \cdot \boldsymbol{\nu_2}$.
`VecBarrettMod` Element-wise modulo q on a vector, $\texttt{VecBarrettMod}(\boldsymbol{\nu}) = \boldsymbol{\nu} \bmod q$, using Barrett Reduction (Barrett 1986).
`VecMulBarrettMod` Element-wise multiplication of two vectors followed by modulo q, where both vector operands are not known constant vectors, $\texttt{VecMulBarrettMod}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = (\boldsymbol{\nu_1} \cdot \boldsymbol{\nu_2}) \bmod q$.

`VecMulBarrettMod` is composed of `VecMul` and `VecBarrettMod`, $\texttt{VecMulBarrettMod}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}) = \texttt{VecBarrettMod}(\texttt{VecMul}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2}))$.
`Automorphism` $\texttt{Automorphism}(\boldsymbol{\nu_1}, \boldsymbol{\nu_2})[i] = \boldsymbol{\nu_1}[\boldsymbol{\nu_2}[i]]$, where $\boldsymbol{\nu_2}$ stores the indices of the output vector elements in the input vector $\boldsymbol{\nu_1}$. For a cyclic left shift operation r, $\boldsymbol{\nu_2}[i] = i \cdot 5^r \bmod N$. For example, with $N = 2^{14}$ and $r = 1, \boldsymbol{\nu_2} = (0, 3277, 6554, 9831, 13108, \cdots)$.
`VecNttMontMod` Element-wise modulo q operation on the resulting vector of a matrix–vector multiplication, where the matrix is in Montgomery form. It is used to apply a modular reduction to the output of NTTU, which will be described in Sects. "Using decomposition method to compute NTT" and "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline".

Algorithms 1 to 6 provide descriptions of the vector operations using the RISC-V vector extension instructions.

In LP-HENN, the vector elements are within 30 bits, as the parameter B is set to 30. This configuration meets the precision requirements outlined in (Kim et al. 2023; Samardzic et al. 2022) while simultaneously optimizing computational throughput. As discussed in Sect. "Vector unit: structure and workflow", the computational throughput of Ara for processing 32-bit elements is twice that of processing 64-bit elements. Setting $B > 32$ would halve the computation speed of the vector unit. So there is no need to distinguish between signed and unsigned integer instructions for the operations. `VecAdd` can be directly implemented using the `vadd` instruction, while `VecMul` can be directly implemented using the `vwmul` instruction.

Algorithm 1 `VecAddMod`

Input: $\boldsymbol{v_1}$ (in v1 register), $\boldsymbol{v_2}$ (in v2 register), q
Output: $\boldsymbol{v}_{\text{out}}$ (in v3 register)
// $\boldsymbol{v}_{\text{out}} \leftarrow \boldsymbol{v_1} + \boldsymbol{v_2}$
1 `vadd.vv v3, v1, v2`
// **if** $\boldsymbol{v}_{out} \geq q$ **then** $\boldsymbol{v}_{\text{out}} \leftarrow \boldsymbol{v}_{\text{out}} - q$
2 `vmsgtu.vi v0, v3, q`
3 `li x1, q`
4 `vsub.vx v3, v3, x1, v0.t`

Algorithm 2 `VecSubMod`

Input: $\boldsymbol{v_1}$ (in v1 register), $\boldsymbol{v_2}$ (in v2 register), q
Output: $\boldsymbol{v}_{\text{out}}$ (in v3 register)
// $\boldsymbol{v}_{\text{out}} \leftarrow \boldsymbol{v_1} - \boldsymbol{v_2}$
1 `vsub.vv v3, v1, v2`
// **if** $\boldsymbol{v}_{out} < 0$ **then** $\boldsymbol{v}_{\text{out}} \leftarrow \boldsymbol{v}_{\text{out}} + q$
2 `vmslt.vx v0, v3, 0`
3 `li x1, q`
4 `vadd.vx v3, v3, x1, v0.t`

Algorithm 3 `VecMulMontMod`

Input: $\boldsymbol{v_1}$ (in v1 register), $\boldsymbol{v_2}$ (in v2 register), q
Output: $\boldsymbol{v}_{\text{out}}$ (in v3 register)
Precompute: $R = 2^{32}$, $R' = R^{-1} \bmod q$, $q' = (RR' - 1)/q \bmod q$ (in x2 register)

```
1  vsetvli x0, x0, e32, m1, ta, ma
2  vwmul.vv v3, v1, v2 // v3 (64): v1 ⊙ v2
3  vnsrl.wi v5, v3, 32 // v5 (32): (v1 ⊙ v2) mod R
4  li x1, q'
   // v5(32): m = ((v1 ⊙ v2) mod R)q' mod R
5  vmul.vx v5, v5, x1
6  li x1, q
7  vwmul.vx v6, v5, x1 // v6 (64): m · q
8  vsetvli x0, x0, e64, m2, ta, ma
9  vadd.vx v6, v3, v6 // v6 (64): (v1 ⊙ v2) + m · q
10 vsetvli x0, x0, e32, m1, ta, ma
11 vnsrl.wi v3, v6, 32 // v3 (32): t = ((v1 ⊙ v2) + m · q)/R
```

Algorithm 4 `VecBarrettMod`

Input: v (64-bit, in v1 register), q (64-bit, in x2 register)
Output: $\mathbf{v_{out}}$ (32-bit, in v2 register)
Precompute: $r = \lfloor \frac{2^{64}}{q} \rfloor$ (64-bit, in x1 register)

```
1 vsetvli x0, x0, e64, m1, ta, ma
2 vmulh.vx v2, v1, x1 // v2(64):m = ⌊v·r / 2^2k⌋
3 vnmsub.vx v2, x2, v1 // v2(64):t = v − m · q
4 vsetvli x0, x0, e32, m1, ta, ma
5 vnsrl.wi v2, v2, 0  // v2: 64->32
  // if t ≥ q then t = t − q
6 vmslt.vx v0, v2, x2
7 vsub.vx v2, v2, x2, v0.t  // v2(32): t
```

Algorithm 5 `Automorphsim`

Input: $\boldsymbol{v_1}$ (in memory, memory address in x1 register), $\boldsymbol{v_2}$ (in memory, memory address in x2 register)
Output: $\boldsymbol{v}_{\text{out}}$ (in v2 register)

```
1 vle32.v v1, (x2)
2 vloxei32.v v2, (x1), v1
```

Algorithm 6 `VecNttMontMod`

Input: $\boldsymbol{v}$ (128-bit, higher part in `v2` register, lower part in `v1` register), q (in `x2` register)
Output: $\boldsymbol{v}_{out}$ (in `v2` register)
Precompute: $R = 2^{64}$, $R' = R^{-1} \bmod q$, $q' = (RR' - 1)/q \bmod q$ (in `x1` register)

```
1 vsetvli x0, x0, e64, m1, ta, ma
2 vmul.vx v3, v1, x1 // m = (v mod R)q' mod R
  // t = (v + mq)/R
3 vmul.vx v4, v3, x2 // lower part of mq
4 vmadc.vv v0, v4, v1
5 vmulh.vx v4, v3, x2 // higher part of mq
6 vadc.vm v2, v1, v4, v0
7 vsetvli x0, x0, e32, m1, ta, ma
8 vnsrl.wi v2, v2, 32
```

Implementations of CKKS operations

This section presents how the primitive operations in CKKS, as well as the operations that compose these primitive operations, such as `KeySwitch` and `ModUp`, are constructed from the basic vector operations described in Sect. ""Basic vector operations and their implementations".

Algorithms 7 to 12 provide the details of the CKKS primitive operations listed in Table 2. Algorithms 13 to 16 describe the FHE operators that constitute these primitive operations, as discussed in Sect. "Hierarchical description of RNS-CKKS".

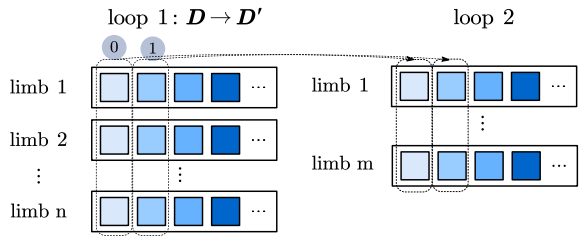


Fig. 7 Computation order of chunks in `BConv`

For primitive operations such as PAdd, PMult, and HAdd, the input ciphertext (with level ℓ) is a pair of polynomials. As discussed in Sect. "RNS-CKKS", the DCRT representation of a polynomial is an $\ell \times N$ matrix. Consequently, the ciphertext comprises a pair of $\ell \times N$ matrices, where each row of these matrices represents the limbs under the moduli $q_1, \ldots, q_\ell$. As described in Sect. "CKKS scheme", the plaintext is a polynomial that also adopts the DCRT representation. Its shape is identical to the ciphertext matrices, i.e., an $\ell \times N$ matrix, where each row represents the limbs under the moduli $q_1, \ldots, q_\ell$.

For operators such as `BConv`, `ModUp`, `ModDown`, and `KeySwitch`, which constitute the primitive operations, the input is the DCRT representation of a polynomial.

Algorithm 7 PAdd

Input: $(\boldsymbol{A}, \boldsymbol{B})$ (ciphertext), $\boldsymbol{P}$ (plaintext)
Output: $(\boldsymbol{A}', \boldsymbol{B}')$ (output ciphertext)
for $j \leftarrow 1$ **to** ℓ **do**
  $\boldsymbol{A}'_j \leftarrow \texttt{VecAddMod}(\boldsymbol{A}_j, \boldsymbol{P}_j, q_j)$
  $\boldsymbol{B}'_j \leftarrow \texttt{VecAddMod}(\boldsymbol{B}_j, \boldsymbol{P}_j, q_j)$
end

Algorithm 8 PMult

Input: $(\boldsymbol{A}, \boldsymbol{B})$ (ciphertext), $\boldsymbol{P}$ (plaintext)
Output: $(\boldsymbol{A}', \boldsymbol{B}')$ (output ciphertext)
for $j \leftarrow 1$ **to** ℓ **do**
  $\boldsymbol{A}'_j \leftarrow \texttt{VecMulBarrettMod}(\boldsymbol{A}_j, \boldsymbol{P}_j, q_j)$
  $\boldsymbol{B}'_j \leftarrow \texttt{VecMulBarrettMod}(\boldsymbol{B}_j, \boldsymbol{P}_j, q_j)$
end

Algorithm 9 HAdd

Input: $(\boldsymbol{A}, \boldsymbol{B})$ (ciphertext 1), $(\boldsymbol{A}', \boldsymbol{B}')$ (ciphertext 2)
Output: $(\boldsymbol{A}'', \boldsymbol{B}'')$ (output ciphertext)
for $j \leftarrow 1$ **to** ℓ **do**
  $\boldsymbol{A}''_j \leftarrow \texttt{VecAddMod}(\boldsymbol{A}_j, \boldsymbol{A}'_j, q_j)$
  $\boldsymbol{B}''_j \leftarrow \texttt{VecAddMod}(\boldsymbol{B}_j, \boldsymbol{B}'_j, q_j)$
end

Algorithm 10 HMult

Input: $\mathrm{ct}_0 = (\boldsymbol{A_0}, \boldsymbol{B_0})$, $\mathrm{ct}_1 = (\boldsymbol{A_1}, \boldsymbol{B_1})$
$\boldsymbol{D}_0 \leftarrow \texttt{VecMulBarrettMod}(\boldsymbol{B}_0, \boldsymbol{B}_1)$
$\boldsymbol{D}_1 \leftarrow \texttt{VecAddMod}(\texttt{VecMulBarrettMod}(\boldsymbol{A}_0, \boldsymbol{B}_1), \texttt{VecMulBarrettMod}(\boldsymbol{A}_1, \boldsymbol{B}_0))$
$\boldsymbol{D}_2 \leftarrow \texttt{VecMulBarrettMod}(\boldsymbol{A}_0, \boldsymbol{A}_1)$
$(\boldsymbol{C}_0, \boldsymbol{C}_1) \leftarrow \mathrm{KeySwitch}(\boldsymbol{D}_2, \mathbf{evk})$
return $(\texttt{VecAddMod}(\boldsymbol{D_0} + \boldsymbol{C_0}), \texttt{VecAddMod}(\boldsymbol{D_1}, \boldsymbol{C_1}))$

Algorithm 11 HRotate

Input: $\mathrm{ct} = (\boldsymbol{A}, \boldsymbol{B})$
$\boldsymbol{A}' \leftarrow \texttt{Automorphsim}(\boldsymbol{A})$
$\boldsymbol{B}' \leftarrow \texttt{Automorphsim}(B)$
$(\boldsymbol{C}_0, \boldsymbol{C}_1) \leftarrow \texttt{KeySwitch}(\boldsymbol{A}', \mathbf{rotkey})$
return $(\boldsymbol{C}_0, \texttt{VecAddMod}(\boldsymbol{B}', \boldsymbol{C}_1))$

Algorithm 12 Rescale

Input: $\mathbf{ct} = (\boldsymbol{D}_0, \boldsymbol{D}_1)$
Output: $(\boldsymbol{O}_0, \boldsymbol{O}_1)$
for $j \leftarrow 0$ **to** 1 **do**
  $\boldsymbol{v}_0 \leftarrow \mathrm{INTT}_{q_\ell}(\boldsymbol{D}_{j\ell})$
  for $i \leftarrow 1$ **to** $\ell - 1$ **do**
    $\boldsymbol{v}_1 \leftarrow \mathrm{NTT}_{q_i}(\boldsymbol{v}_0)$
    $\boldsymbol{O}_{ji} \leftarrow$ `VecSubMod`$(D_{ji}, \boldsymbol{v}_1)$
  end
end

Algorithm 13 `KeySwitch`

Input: $\boldsymbol{D}$, $\mathbf{evk} = ((\mathrm{evk}_{0,0}, \mathrm{evk}_{0,1}), \ldots, (\mathrm{evk}_{k+L+1,0}, \mathrm{evk}_{k+L+1,1}))$
Output: $(\boldsymbol{O}, \boldsymbol{O}')$
// `DComp`
for $i \leftarrow 1$ **to** ℓ **do**
  $g \leftarrow \lfloor (i-1)/\alpha \rfloor + 1$
  // $\alpha = \lceil \frac{L}{\mathtt{dnum}} \rceil$, $\beta = \lceil \frac{\ell}{\alpha} \rceil$, $Q' = \prod_{j=\ell+1}^{\alpha\beta} q_j$, $Q_j = \prod_{l=j\alpha+1}^{(j+1)\alpha} q_l$
  $\boldsymbol{D}'_i \leftarrow$ `VecMulMontMod`$(\boldsymbol{D}_i, [Q']_{q_i}[(\prod_{j=1}^{\beta} Q_j)/Q_g]_{q_i})$
end
$d'' \leftarrow$ `ModUp`$(\mathbf{d}', \overbrace{\mathbf{0}}^{\alpha\beta-(\ell+1)})$
// `InnerProduct`
$\mathrm{sum}_0, \mathrm{sum}_1 \leftarrow \mathbf{0}$
for $i \leftarrow 0$ **to** $k + \alpha\beta - 1$ **do**
  for $j \leftarrow 0$ **to** 1 **do**
    $\mathrm{sum} \leftarrow$ `VecAdd`$(\mathrm{sum},$ `VecMul`$(d''_i, \mathrm{evk}_{i,j}))$
  end
end
for $i \leftarrow 0$ **to** m **do**
  $\mathrm{sum}_i \leftarrow$ `VecBarrettMod`(sum_i)
  $\mathrm{output}_i \leftarrow$ `ModDown`(sum_i)
end

Algorithm 14 `ModUp`

Input: $\boldsymbol{D}_{\{q_{g\cdot\alpha+1},\ldots,q_{(g+1)\cdot\alpha}\}}$
Output: $\boldsymbol{O}_{\{p_1,\ldots,p_k,q_1,\ldots,q_{\alpha\beta}\}}$
for $i \leftarrow 1$ **to** α **do**
  $\tilde{\boldsymbol{D}}_i \leftarrow \mathrm{INTT}_{q_{g\cdots\alpha+i}}(\boldsymbol{D}_i)$
end
$\tilde{\boldsymbol{T}} \leftarrow \texttt{BConv}(\tilde{\boldsymbol{D}}, \{p_1,\ldots,p_k\})$ `// T has` k `limbs under` $\{p_1,\ldots,p_k\}$
for $i \leftarrow 1$ **to** k **do**
  $\boldsymbol{T}_i \leftarrow \mathrm{INTT}_{p_i}(\tilde{\boldsymbol{T}}_i)$
end
for $i \leftarrow 1$ **to** k **do**
  $\boldsymbol{O}_i \leftarrow \boldsymbol{T}_i$
end
for $j \leftarrow 1$ **to** ℓ **do**
  $\boldsymbol{O}_{k+j} \leftarrow \boldsymbol{D}_j$
end

Algorithm 15 `ModDown`

Input: $\boldsymbol{D}_{\{p_1,\ldots,p_k,q_1,\ldots,q_\ell\}}$
Output: $\boldsymbol{O}_{\{q_1,\ldots,q_\ell\}}$
for $i \leftarrow 1$ **to** k **do**
  $\tilde{\boldsymbol{D}}_i \leftarrow \mathrm{NTT}_{p_i}(\boldsymbol{D}_i)$
end
$\tilde{\boldsymbol{T}} \leftarrow \mathrm{BConv}(\tilde{\boldsymbol{D}}, \{q_1,\ldots,q_\ell\})$
for $i \leftarrow 1$ **to** ℓ **do**
  $\boldsymbol{T}_i \leftarrow \mathrm{INTT}_{q_i}(\tilde{\boldsymbol{T}}_i)$
  $\boldsymbol{O}_i \leftarrow \texttt{VecSubMod}(\boldsymbol{D}_i, \boldsymbol{T}_i)$
end

Algorithm 16 `BConv`

Input: $\boldsymbol{D}_{\{q_1,\ldots,q_m\}}$, target basis $\{p_1,\ldots,p_n\}$
Output: $\boldsymbol{O}_{\{p_1,\ldots,p_n\}}$
`// loop 1`
for $i \leftarrow 1$ **to** m **do**
  $\boldsymbol{D}'_i \leftarrow \texttt{VecMulMontMod}(\boldsymbol{D}_i, [Q_i^{-1}]_{q_i})$ `//` $Q_i = (\prod_{j=0}^{m} q_j)/q_i$
end
`// loop 2`
for $j \leftarrow 1$ **to** n **do**
  $\boldsymbol{v} \leftarrow \boldsymbol{0}$
  for $i \leftarrow 1$ **to** m **do**
    `// lazy modular reduction`
    $\boldsymbol{t} \leftarrow \texttt{VecMul}(\boldsymbol{D}'_i, [Q_i]_{p_j})$
    $\boldsymbol{v} \leftarrow \texttt{VecAdd}(\boldsymbol{v}, \boldsymbol{t})$
  end
  $\boldsymbol{O}_j \leftarrow \texttt{VecBarrettMod}\ (\boldsymbol{v})$
end

Chunk&fusion optimization

Performance of FHE accelerators is often constrained by memory bandwidth (Kim et al. 2022a). Minimizing data movement between the cache and SPM, as well as between the VRF and cache, significantly impacts the performance of LP-HENN. This section proposes the chunk&fusion optimization to reduce data movement and enhance performance.

Fusion refers to the thorough integration of computations within the same FHE primitive operation, ensuring that after reading data, as much computation as possible is performed to eliminate all unnecessary write-after-read operations.

According to Sect. "Vector unit: structure and workflow", due to the limitation of VLEN, the maximum vector operand length supported by the vector unit is much smaller than the vector length in FHE applications. The VLEN of Ara when #channels=16 is 16384 bits. In contrast, when $B = 30, N = 8192$, the size of one limb is 32768 bytes. Even with an LMUL of 8, a single vector instruction can process at most 4096 32-bit elements.

Consequently, the vector unit processes only N_1 (which is 128 according to Sect. "Using decomposition method to compute NTT") elements of a limb at a time, referred to as a chunk. The operands of operators such as `VecAddMod` in Sect. "Hierarchical organization of NTTU" become a chunk instead of an entire limb. A chunk does not consist of 128 consecutive vector elements in a limb, which will be elaborated upon in Sect. "Data layout".

Due to the limited capacity of the VRF, vector registers should be released as early as possible for subsequent computations. There are three cases for releasing a chunk:

1. The chunk is no longer required for subsequent computations.
2. The chunk is needed for subsequent computations, but due to the VRF capacity limitation, it cannot be stored until the next use. In this case, it needs to be written back to the SPM, and the vector register is released.
3. NTTU (Sect. "NTT Unit") requires the chunk as input. In this case, it needs to be written back to the SPM for NTTU to read. The vector register should be released because the computations after NTT will not use the original chunk data but rather the output of NTT.

The chunk&fusion optimization is illustrated using HMult as an example. Algorithm 17 presents the computation of HMult from the beginning to `DComp` in `KeySwitch` using the chunk&fusion optimization. After computing $\boldsymbol{D}_0$, $\boldsymbol{D}_1$, and $\boldsymbol{D}_2$, the chunks corresponding to $\boldsymbol{A}_0$, $\boldsymbol{A}_1$, $\boldsymbol{B}_0$, and $\boldsymbol{B}_1$ are no longer needed, and their vector registers are released, corresponding to case 1.

According to Algorithm 10, $\boldsymbol{D}_0$ and $\boldsymbol{D}_1$ will be used after `KeySwitch` is completed. However, since NTT is performed on `DComp` of $\boldsymbol{D}_2$ in `KeySwitch`, and NTT requires entire limbs of $\boldsymbol{D}_2$, $\boldsymbol{D}_0$ and $\boldsymbol{D}_1$ are also computed before the NTT ($\boldsymbol{D}_0$, $\boldsymbol{D}_1$, and $\boldsymbol{D}_2$ depend on the same inputs $\boldsymbol{A}_0$, $\boldsymbol{A}_1$, $\boldsymbol{B}_0$, and $\boldsymbol{B}_1$, so they are computed together). However, the vector registers cannot hold the complete $\boldsymbol{D}_0$ and $\boldsymbol{D}_1$, so they need to be written back immediately after the computation of $\boldsymbol{D}_0$ and $\boldsymbol{D}_1$ is completed, corresponding to case 2.

Due to the fusion optimization, for $\boldsymbol{D}_2$, its chunks are not written back immediately after Line 3 in 10, but rather after the `DComp` computation is completed. This processing order differs from that of Algorithm 10. Instead of starting the `DComp` computation after completing the `VecMulBarrettMod` computation for the entire limb of d_2, the same computation is performed for the next chunk after completing the `VecMulBarrettMod` and `DComp` computations for one chunk. The chunk needs to be written back after `DComp` because the next operation following `DComp` is NTT in `ModUp`, corresponding to case 3.

The `BConv` algorithm fully demonstrates the performance gains brought by chunk&fusion optimizations. In `loop 1` of `BConv` in Algorithm 16, the obtained $\boldsymbol{D}'$ is repeatedly read n times in `loop 2`. The computation order of chunks in `BConv` optimized with chunk&fusion is shown in Fig. 7. In `loop 1`, the chunks at the same position of each limb of $\boldsymbol{D}$ are computed first, and the results are saved in vector registers. Then, the chunks at the same position of all limbs of the output $\boldsymbol{O}$ are computed. After the computation is completed, the chunks of $\boldsymbol{D}'$ are no longer needed, and the vector registers are released. For `loop 2`, since the next step of `BConv` is NTT (Algorithm 14, 15), the chunk of each limb of $\boldsymbol{O}$ should be written back immediately and the vector registers released. Then, the same computation process is repeated for chunks at new positions. Algorithm 18 provides the algorithmic description of `BConv` optimized with chunk&fusion. Compared to the unoptimized `BConv` algorithm (Algorithm 16), Algorithm 18 loads $\boldsymbol{D}'$ into vector registers only once.

Experiments show that compared to using the unoptimized `BConv`, employing `BConv` optimized with chunk&fusion reduces the overall data movement between the VRF and L1 D-cache of the LoLa-CIFAR application by 33.32%, demonstrating that chunk&fusion optimization significantly reduces data movement.

Algorithm 17 HMult with chunk&fusion optimization

Input: $\mathbf{ct}_0 = (\boldsymbol{A}_0, \boldsymbol{B}_0)$, $\mathbf{ct}_1 = (\boldsymbol{A}_1, \boldsymbol{B}_1)$
for $i \leftarrow 1$ **to** ℓ **do**
  $g \leftarrow \lfloor (i-1)/\alpha \rfloor$
  for $ic \leftarrow 0$ **to** N_2 // N_2 is number of chunks in a limb
  do
    load $\boldsymbol{A}_{0i,\mathrm{ic}}$, load $\boldsymbol{A}_{1i,\mathrm{ic}}$, load $\boldsymbol{B}_{0i,\mathrm{ic}}$, load $\boldsymbol{B}_{1i,\mathrm{ic}}$
    $\boldsymbol{D}_{0i,\mathrm{ic}} \leftarrow$ VecMulBarrettMod($\boldsymbol{B}_{0i,ic}$, $\boldsymbol{B}_{1i,ic}$)
    store $\boldsymbol{D}_{0i,\mathrm{ic}}$ // so the vector register can be reused
    $\boldsymbol{D}_{1i,\mathrm{ic}} \leftarrow$ VecAddMod(VecMulBarrettMod($\boldsymbol{A}_{0i,ic}$, $\boldsymbol{B}_{1i,ic}$), VecMulBarrettMod($\boldsymbol{A}_{1i,ic}$, $\boldsymbol{B}_{0i,ic}$))
    store $\boldsymbol{D}_{1i,\mathrm{ic}}$ // the vector register can be reused
    $\boldsymbol{D}_{2i,\mathrm{ic}} \leftarrow$ VecMulBarrettMod($\boldsymbol{A}_{0i,ic}$, $\boldsymbol{A}_{1i,ic}$)
    // now vector registers of $\boldsymbol{A}_{0i,\mathrm{ic}}$, $\boldsymbol{A}_{1i,\mathrm{ic}}$, $\boldsymbol{B}_{0i,\mathrm{ic}}$, $\boldsymbol{B}_{1i,\mathrm{ic}}$ can be reused
    $\boldsymbol{D}'_{2i,\mathrm{ic}} \leftarrow$ VecMulMontMod($\boldsymbol{D}_{2i,ic}$, $(\prod_{j=\ell+1}^{\alpha\beta-1} q_j)(\prod_{j=0}^{\beta-1} Q_j)/Q_g$) // DComp
    store $\boldsymbol{D}_{2i,\mathrm{ic}}$ // input of NTT, needed by NTTU
  end
end

Algorithm 18 BConv with chunk&fusion optimization

Input: $\boldsymbol{D}_{\{q_1,\ldots,q_m\}}$, target basis $\{p_1, \ldots, p_n\}$
Output: $\boldsymbol{O}_{\{p_1,\ldots,p_n\}}$
for $ic \leftarrow 0$ **to** $N_2 - 1$ // N_2 is number of chunks in a limb
do
  // loop 1
  for $i \leftarrow 1$ **to** m **do**
    $\boldsymbol{D}'_{i,\mathrm{ic}} \leftarrow$ VecMulMontMod($\boldsymbol{D}_{i,ic}$, $[Q_i^{-1}]_{q_i}$) // $Q_i = (\prod_{j=1}^{m} q_j)/q_i$
  end
  // loop 2
  for $j \leftarrow 1$ **to** n **do**
    $\boldsymbol{v} \leftarrow \mathbf{0}$
    for $i \leftarrow 1$ **to** m **do**
      // lazy modular reduction
      $\boldsymbol{t} \leftarrow$ VecMul($\boldsymbol{D}'_{i,ic}$, $[Q_i]_{p_j}$) // use the same register over loop
      $\boldsymbol{v} \leftarrow$ VecAdd($\boldsymbol{v}$, $\boldsymbol{t}$)
    end
    $\boldsymbol{O}_{j,\mathrm{ic}} \leftarrow$ VecBarrettMod ($\boldsymbol{v}$)
    store $\boldsymbol{O}_{j,\mathrm{ic}}$ // the register can be reused
    // registers storing $\boldsymbol{D}'_{i,\mathrm{ic}}$ i from 1 to m can be released
  end
end

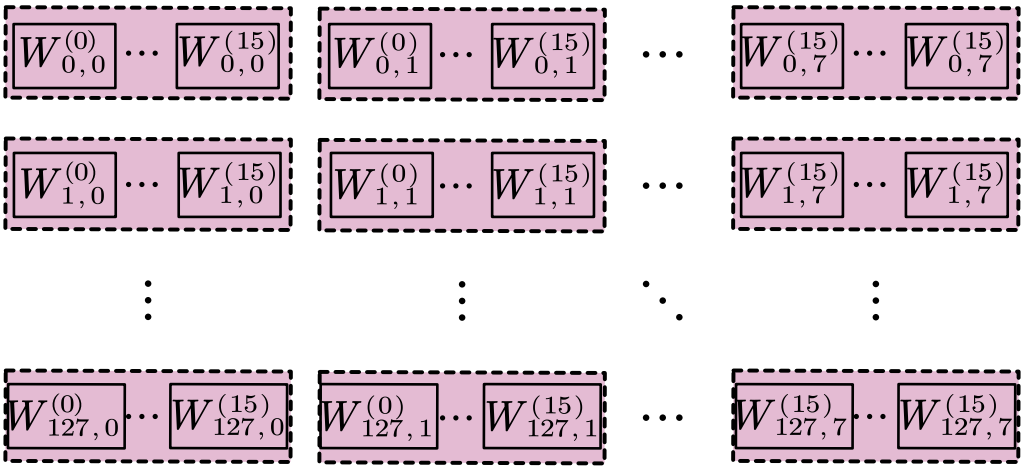


Fig. 8 A 128Ã-128 crossbar stores 128Ã-8 twiddle factors

NTT Unit

Using array of crossbars to compute NTT

Section "Matrix-vector multiplication accelerator based on ReRAM crossbars" describes how an mÃ-n ReRAM crossbar simultaneously computes the dot products of n vectors of length m. However, due to the practical limitations of ReRAM technology, two adjustments need to be made to ensure its feasibility.

Adjustment 1: Storing only 1 bit per cell to avoid errors The resistances of ReRAM are susceptible to random offsets and discrepancies from the target resistances due to the manufacturing process and the underlying electrochemical mechanisms. Even within the same device, resistances may exhibit slight variations across different cycles, which can result in current errors during dot product computations. If the current error exceeds the ADC resolution, the dot product calculation results will be erroneous. Unlike neural network accelerators that exhibit tolerance to errors (Shafiee et al. 2016), NTT mandates strict accuracy. For example, let $(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{N-1})$ be $\mathrm{NTT}(\boldsymbol{a})$, $\mathrm{INTT}(\tilde{a}_0, \tilde{a}_1 - 1, \ldots, \tilde{a}_{N-1})$ will deviate significantly from $(a_0, a_1, \ldots, a_{N-1})$. Therefore, for LP-HENN, it is necessary to ensure the accuracy of crossbar computations.

Prior studies (Chou et al. 2019) have demonstrated that multi-level cell (MLC) requires the use of more complex DACs and ADCs, and can be more easily affected by noise and process variation. To minimize the likelihood of errors, NTTU adopts the approach of RM-NTT (Park et al. 2022), wherein each cell in the crossbar stores only one bit. A B-bit twiddle factor is stored in B adjacent cells in a row. Thus, a crossbar of size $c \times c$ can store $c \times \lfloor \frac{c}{B} \rfloor$ twiddle factors. Figure 8 illustrates the mapping of 128Ã-8 twiddle factors ($B = 16$) within a 128Ã-128 crossbar, where $\boldsymbol{W}_{i,j}^{(b)}$ represents the b-th bit of $\boldsymbol{W}_{i,j}$.

Adjustment 2: Using serial input to minimize ADC resolution bits ADC is the component with the largest area and power consumption (Shafiee et al. 2016; Park et al. 2022). For example, in RM-NTT, the ADC accounts for 77.7% of the total power consumption (Park et al. 2022). The power consumption and area overhead of ADC grow exponentially with the ADC resolution bits (Shafiee et al. 2016). Therefore, minimizing the ADC resolution bits is crucial for reducing power consumption and area. The ADC resolution bits grow logarithmically with the product of the crossbar size c, the bit width of the input vector, and the number of levels of each cell (which is $2^1 = 2$ as described in Adjustment 1).

To control the ADC resolution bits, NTTU adopts a serial input approach, where only one bit of each element in the input vector is input in each cycle. Computing an B-bit NTT requires B crossbar operations, and the input vector at cycle t is $(x_0^{(t)}, x_1^{(t)}, \ldots, x_{n-1}^{(t)})$. For example, with $N = 128$ and $B = 16$, the input to the crossbar at cycle 0 is $(x_0^{(0)}, x_1^{(0)}, \ldots, x_{127}^{(0)})$, the input at cycle 1 is $(x_0^{(1)}, x_1^{(1)}, \ldots, x_{127}^{(1)})$, and the input at cycle 15 is $(x_0^{(15)}, x_1^{(15)}, \ldots, x_{127}^{(15)})$. This approach ensures that the input bit length for a single crossbar dot product operation is 1 bit, thereby minimizing the ADC resolution bits.

Taking the first column of twiddle factors $(\boldsymbol{W}_{0,0}, \boldsymbol{W}_{1,0}, \cdots, \boldsymbol{W}_{127,0})$ with $N = 128$ and $B = 16$ as an example, we illustrate how to combine the outputs of multi-cycle crossbars through shifting and addition to obtain the dot product result of $(\boldsymbol{W}_{0,0}, \boldsymbol{W}_{1,0}, \cdots, \boldsymbol{W}_{127,0})$ and the input vector x. $(\boldsymbol{W}_{0,0}, \boldsymbol{W}_{1,0}, \cdots, \boldsymbol{W}_{127,0})$ will be stored in the first 16 columns of cells in the crossbar.

At cycle t, the inner product result obtained by the j-th column of the crossbar is $\sum_{i=0}^{127} \boldsymbol{W}_{i,0}^{(j)} x_i^{(t)}$. By shifting and summing these 16-column results, where the j-th column is left-shifted by j bits:

$$\begin{aligned}\sum_{j=0}^{15}\left(\sum_{i=0}^{127} \boldsymbol{W}_{i,0}^{(j)} x_i^{(t)}\right)2^j &= \sum_{j=0}^{15}\left(\sum_{i=0}^{127}\left(\boldsymbol{W}_{i,0}^{(j)}2^j\right)x_i^{(t)}\right)\\ &= \sum_{i=0}^{127}\left(\sum_{j=0}^{15}\left(\boldsymbol{W}_{i,0}^{(j)}2^j\right)\right)x_i^{(t)}\\ &= \sum_{i=0}^{127}\boldsymbol{W}_{i,0}x_i^{(t)}\end{aligned}$$

Then, by shifting and summing the results of each cycle, where the result of cycle t is left-shifted by t bits:

$$\begin{aligned}\sum_{t=0}^{15}\left(\sum_{i=0}^{127}\boldsymbol{W}_{i,0}x_i^{(t)}\right)2^t &= \sum_{i=0}^{127}\boldsymbol{W}_{i,0}\left(\sum_{t=0}^{15}x_i^{(t)}2^t\right)\\ &= \sum_{i=0}^{127}\boldsymbol{W}_{i,0}x_i\end{aligned}$$

Therefore, the dot product of twiddle factors and the 16-bit input vector can be obtained through shifting and addition.

Figure 9 illustrates the collaborative computation of NTT by multiple crossbars, with $N = 512$, $B = 16$, and $c = 128$. A single crossbar can store 128×8 twiddle factors, necessitating a total of 256 crossbars to store all the twiddle factors. The 256 crossbars are arranged in a 4Ã-64 crossbar array. The crossbars in the same row of the array receive the same input. The 4 crossbars in the same column are responsible for computing the dot product of 8 columns of twiddle factors and the input, with each crossbar responsible for the dot product of a sub-vector of length 128. For example, at cycle 0, the input to crossbar $C_{0,0}$ is $(x_0^{(0)}, x_1^{(0)}, \ldots, x_{127}^{(0)})$, the input to crossbar $C_{1,0}$ is $(x_{128}^{(0)}, x_{129}^{(0)}, \ldots, x_{255}^{(0)})$, the input to crossbar $C_{2,0}$ is $(x_{256}^{(0)}, x_{257}^{(0)}, \ldots, x_{383}^{(0)})$, and the input to crossbar $C_{3,0}$ is $(x_{384}^{(0)}, x_{385}^{(0)}, \ldots, x_{511}^{(0)})$. The sum of the results of the 4 crossbars in the same column is the dot product of the mapped 8 columns of twiddle factors and the input vector. Considering the first column of twiddle factors as an example:

$$\sum_{j=0}^{3}\sum_{i=0}^{127}\boldsymbol{W}_{128j+i,0}x_{128j+i} = \sum_{i=0}^{511}\boldsymbol{W}_{i,0}x_i$$

Using decomposition method to compute NTT

LP-HENN adopts the following optimizations from RM-NTT: **Sharing crossbars for INTT and NTT** The twiddle factors of INTT should be the modular inverse of the twiddle factors at the same position in NTT. RM-NTT proposed an optimization that allows NTT and INTT to share the same crossbars, which reduces the number of crossbars by half. Let $(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{N-1})$ be $\mathrm{NTT}_{N,q}(\boldsymbol{a})$ The i-th output of $\mathrm{INTT}_{N,q}\left(\left(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{N-1}\right)\right)$ is $a_i = N^{-1}\sum_{j=0}^{N-1}\psi_{N,q}^{(N-i)\times j}\tilde{a}_j \bmod q$, where $\psi_{N,q}^{(N-i)\times j}$ is a modular inverse of $\psi_{N,q}^{i\times j}$. Let $i' = N - i$, then $a_{N-i'} = N^{-1}\sum_{j=0}^{N-1}\psi_{N,q}^{i'\times j}\tilde{a}_j \bmod q$. Renaming i' to i, we have $a_{N-i} = N^{-1}\sum_{j=0}^{N-1}\psi_{N,q}^{i\times j}\tilde{a}_j \bmod q$. Therefore, INTT can share the same twiddle factors matrix with NTT, and the output vector will be in the reverse order of the original INTT output vector.

Replacing multiplication with shifting in NTT/INTT Computing INTT requires multiplying the dot product result by $N^{-1} \bmod q$. In RM-NTT, the crossbar stores the result of twiddle factors multiplied by N^{-1}, i.e., $N^{-1}\psi_{N,q}^{i\times j} \bmod q$. INTT no longer needs to multiply by $N^{-1} \bmod q$, and NTT can use shifting to multiply by N as N is a power of 2, thus replacing multiplication with shifting.

Storing twiddle factors in Montgomery form The result of the NTT dot product needs to be modulo q. As the twiddle factors are known constants, Montgomery modular multiplication can be used, which requires the known multiplier to be in its Montgomery form. Thus, the crossbar stores the Montgomery form of twiddle factors $\psi_{N,q}N^{-1}R \bmod q$, where R is a constant in the Montgomery algorithm. In RM-NTT, $R = N^2 2^k$, and in LP-HENN, $R = 2^{64}$.

Due to these optimizations, the crossbar stores $\psi_{N,q}^{ij}N^{-1}2^{64} \bmod q$ instead of $\psi_{N,q}^{ij}$.

However, the exponential increase in required hardware resources with the increase of N prevents the use of RM-NTT for FHE. RM-NTT stores all N^2 twiddle factors on the crossbar, and the number of crossbars grows exponentially with the number of bits of N. Although RM-NTT proposed an optimization that utilizes the symmetry of twiddle factors, storing the twiddle factors matrix of $N/2$-point NTT can support N-point NTT, a large number of crossbars are still required when N is large. For example, to support NTT computation with $B = 30, N = 2^{13}$, RM-NTT requires 131072 crossbars of size 128Ã-128. For NTT computation with $B = 30, N = 2^{14}$, it requires 2097152 crossbars of size 128Ã-128, which is impractical.

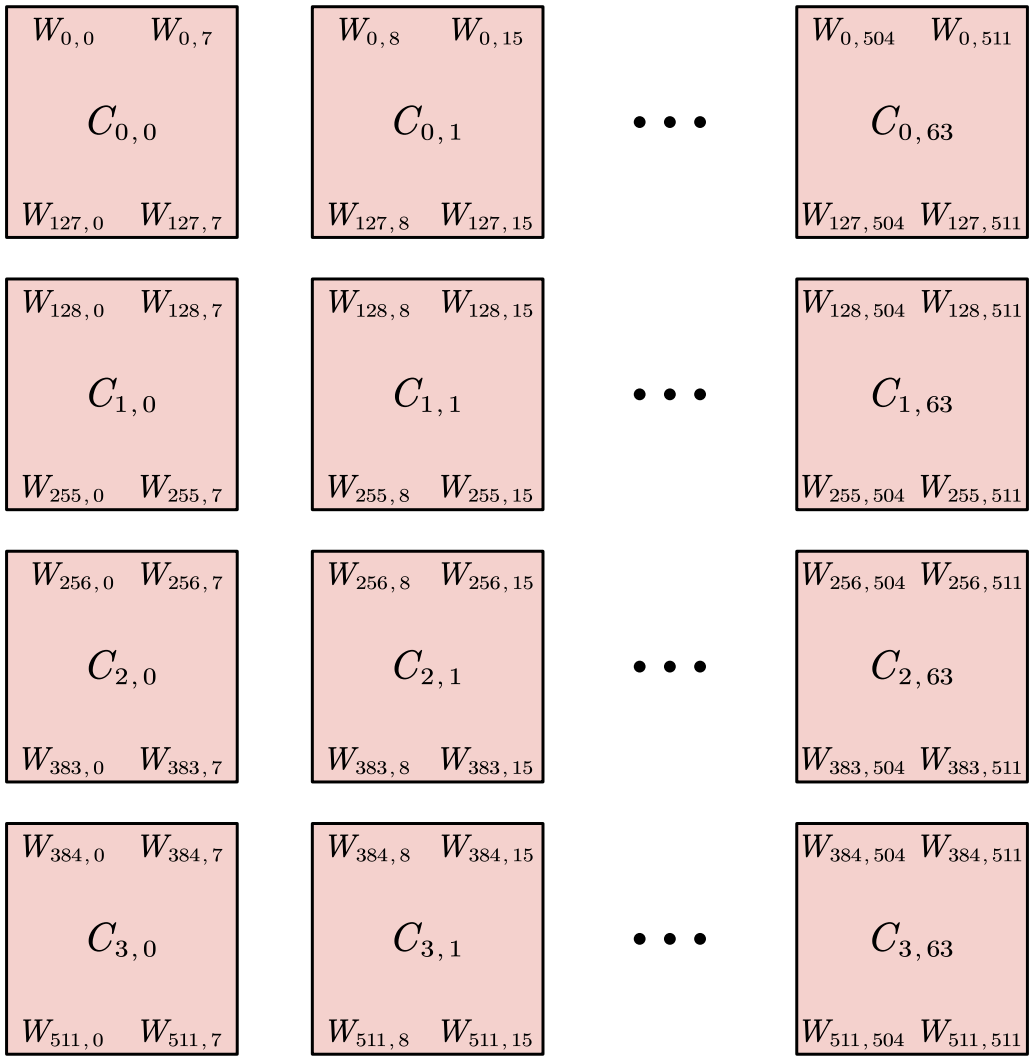


Fig. 9 An array of 256 crossbars for computing 512-point NTT ($B = 16, c = 128$)

To mitigate this challenge, LP-HENN adopts the NTT decomposition method in TensorFHE(Fan et al. 2023). $\boldsymbol{W}_N\boldsymbol{x}$ is decomposed into $\left(\left(\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right) \odot \boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}\right)^T$, where $N_1 \times N_2 = N$, $\boldsymbol{X}$ is the input vector $\boldsymbol{x}$ arranged in a matrix of size $N_1 \times N_2$ by row, $\boldsymbol{W}_{N_1}$ is the twiddle factor matrix of N_1-point NTT, $\boldsymbol{W}_{N_2}$ is the twiddle factor matrix of N_2 -point NTT, and $\boldsymbol{W}_{N,N_1\times N_2}$ is the submatrix of the first N_1 rows and N_2 columns of the twiddle factors matrix of N-point NTT $\boldsymbol{W}_N$. When $N = 8192$, $N_1 = 128$, $N_2 = 64$; when $N = 16384$, $N_1 = 128$, $N_2 = 128$.

Using the decomposition method, the crossbars only need to store $\boldsymbol{W}_{N_1}$ and $\boldsymbol{W}_{N_2}$. For example, when $N = 2^{14}$, $N_1 = N_2 = 2^7$, crossbars only need to store 2 matrices $\boldsymbol{W}_{N_1}$ and $\boldsymbol{W}_{N_2}$ of size 128Ã-128. The stored twiddle factors are only 0.05% of RM-NTT, greatly reducing the number of crossbars required by NTTU.

When $N_1 = N_2 = 128$ and $B = 30$, the computation process of $\left(\left(\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right) \odot \boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}\right)^T$ and the mapping of twiddle factors on crossbars are shown in Fig. 10. The crossbars are divided into two groups, each group forming one processing element (PE), called PE 0 and PE 1. PE 0, consisting of crossbars $(C_{0,0}, C_{0,1}, \ldots, C_{0,31})$, stores $\boldsymbol{W}_{N_1}$ and is responsible for computing $\boldsymbol{W}_{N_1}\boldsymbol{X}$ using the mapping method described in Sect. "Using array of crossbars to compute NTT". PE 1, consisting of crossbars $(C_{1,0}, C_{1,1}, \ldots, C_{1,31})$, stores $\boldsymbol{W}_{N_2}$ and is

responsible for computing the matrix multiplication of $(\boldsymbol{W}_{N_1}\boldsymbol{X}) \odot \boldsymbol{W}_{N,N_1\times N_2}$ and $\boldsymbol{W}_{N_2}$.

The computation is divided into the following 3 stages:

Stage 1 Compute $\boldsymbol{W}_{N_1}\boldsymbol{X}$, which requires N_2 dot product operations. Each cycle, PE 0 uses a column of X $(x_i, x_{i+N_2}, x_{i+2N_2}, \cdots, x_{i+(N_1-1)N_2}), 0 \le i < N_1$ as the input, and the corresponding output is $(o_{0,i,0}, o_{0,i,1}, \ldots, o_{0,i,N_1-1})$. Since input is in serial, a total of N_2B crossbar cycles are required.

Stage 2 Compute $((\boldsymbol{W}_{N_1}\boldsymbol{X}) \odot \boldsymbol{W}_{N,N_1\times N_2}$, which is the element-wise multiplication result of the output of stage 1 and $\boldsymbol{W}_{N,N_1\times N_2}$ modulo q. The input is the output vector $(o_{0,i,0}, o_{0,i,1}, \ldots, o_{0,i,N_1-1})$ of stage 1, and the output is $(o_{1,i,0}, o_{1,i,1}, \ldots, o_{1,i,N_1-1})$. This stage can be pipelined with stage 1.

Stage 3 Compute $\left((\boldsymbol{W}_{N_1}\boldsymbol{X}) \odot \boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}$. The input is all the elements at the same position of all N_2 output vectors of stage 2. Therefore, the computation of this stage can only start after the first two stages are completed and cannot be pipelined.

In summary, although the decomposition method greatly reduces the number of crossbars, it introduces the following problems:

Problem 1 High latency. Stage 3 can only start after the first two stages are completed. Due to the serial input mode of crossbars, stage 1 and stage 3 require a total of $(N_1 + N_2)B$ crossbar cycles. Limited by the read latency, the working cycle of a crossbar is about 40 ns, which is tens of times that of the Ara vector coprocessor. When $N_1 = N_2 = 128$ and $B = 30$, the latency of stage 1 and stage 3 is 307200 ns, which is too high.

Problem 2 Crossbars do not directly support the element-wise multiplication and modulo operation of stage 2.

Section "Hierarchical organization of NTTU" provides a solution to the first problem, and Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline" provides a solution to the second problem.

Hierarchical organization of NTTU

The solution to Problem 1 mentioned in Sect. "Using decomposition method to compute NTT" is to increase the number of crossbars to reduce the latency of the NTTU. Figure 11 illustrates the hierarchical organization of crossbars within the NTTU, comprising tiles, groups, and PEs. Each q_j corresponds to two tiles: tile j and tile $j + L$, responsible for Stage 1 and Stage 3, respectively, resulting in a total of $2L$ tiles in the NTTU. Tile j stores the twiddle factor matrix $\boldsymbol{W}_{N_1,q_j}N^{-1}2^{64} \bmod q_j$, while tile $j + L$ stores the twiddle factor matrix $\boldsymbol{W}_{N_2,q_j}N^{-1}2^{64} \bmod q$. Both Tile j and Tile $j + L$ are responsible for j-th limb $\in \mathbb{Z}_{q_j}^N$. Even if $N_1 = N_2$ and $\boldsymbol{W}_{N_1} = \boldsymbol{W}_{N_2}$, $\boldsymbol{W}_{N_1}$ and $\boldsymbol{W}_{N_2}$ must be stored in separate

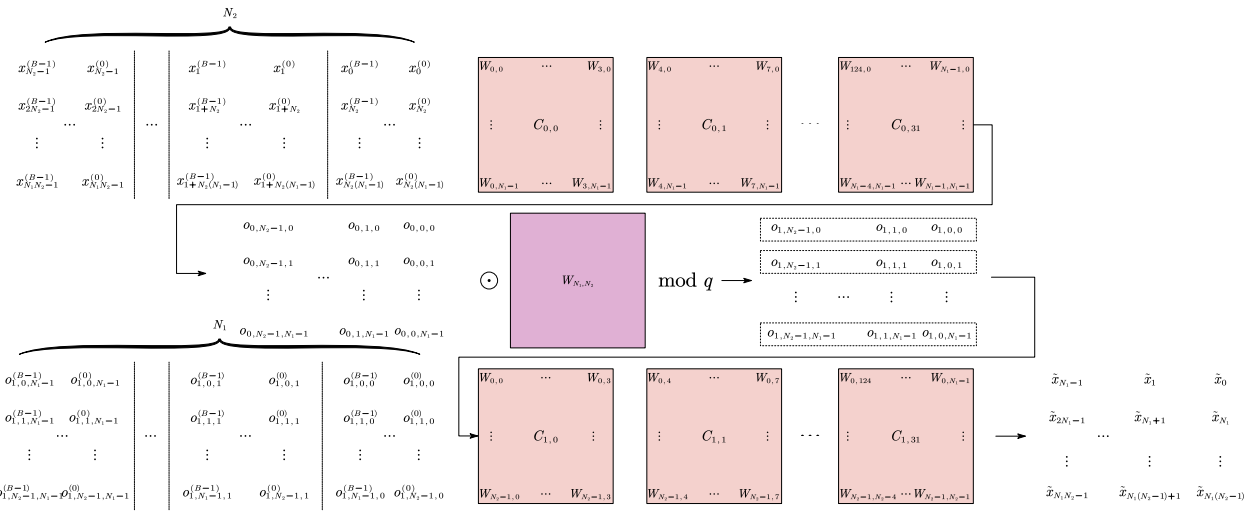


Fig. 10 NTTU uses the decomposition formula $\left(\left((\boldsymbol{W}_{N_1}X) \odot \boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}\right)^T$ to compute NTT, where $N_1 = N_2 = 128$ and $B = 30$

tiles due to their distinct ways of participating in the dot product computations. For $\boldsymbol{W}_{N_1}$, its rows participate in the dot product computations, necessitating the mapping of rows to the columns of the crossbar. Conversely, for $\boldsymbol{W}_{N_2}$, the columns engage in the dot product computations, requiring the mapping of columns to the columns of the crossbar. Since each limb corresponds to a different tile, the NTT computations for each limb can be fully parallelized.

Taking $\boldsymbol{W}_{N_1}$ as an example, let us describe the composition of a tile. As stated in Sect. Using decomposition method to compute NTT", one PE requires N_2B crossbar cycles to compute $\boldsymbol{W}_{N_1}\boldsymbol{X}$. By replicating the PE B times, a group is formed. Different PEs within a group are responsible for computing the matrix–vector multiplication of different bits of the same input vector with $\boldsymbol{W}_{N_1}$. PE i in Tile j takes the i-th bit of the input vector $(x_j^{(i)}, x_{N_2+j}^{(i)}, \ldots, x_{N_2(N_1-1)+j}^{(i)})$ as input. In addition to PEs, a group contains shifters, adders, and output vector registers for shifting and summing the results of each PE, as described in Sect. "Using array of crossbars to compute NTT". A group can complete a matrix–vector multiplication of $\boldsymbol{W}_{N_1}$ in one crossbar cycle, reducing the computation of Stage 1 ($\boldsymbol{W}_{N_1}\boldsymbol{X}$) to N_2 crossbar cycles.

By replicating the group g times, the latency can be further reduced to $\frac{N_2}{g}$ crossbar cycles. As Fig. 11 illustrates, input vectors (different columns of $\boldsymbol{X}$) in Stage 1 are distributed among different groups. Group 0 is responsible for $(x_{ig}, x_{ig+N_2}, \cdots, x_{ig+N_2(N_1-1)}), 0 \le i < N_2/g$, Group 1 is responsible for $(x_{ig+1}, x_{ig+1+N_2}, \cdots, x_{ig+1+N_2(N_1-1)}), 0 \le i < (N_2-1)/g$, and Group $g-1$ is responsible for $(x_{ig+g-1}, x_{ig+g-1+N_2}, \cdots, x_{ig+g-1+N_2(N_1-1)}), 0 \le i < (N_2-g+1)/g$. The analysis of the value of g is presented in Sect. "Number of function units".

In summary, the NTTU comprises $2L$ tiles, with each tile containing g groups, and each group consisting of B PEs. The latency of Stage 1 is reduced from N_2B crossbar cycles to $\frac{N_2}{g}$ crossbar cycles, while the latency of Stage 2 is reduced from N_1B crossbar cycles to $\frac{N_1}{g}$ crossbar cycles.

Collaborative computation process of NTT with NTTU and Vector Unit and pipeline

The solution to Problem 2 in Sect. "Using decomposition method to compute NTT" is to use the vector unit to compute the modulo of the outputs generated by the crossbar and compute Stage 2 $\left(\left(\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right) \odot \boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}\right)^T$, which is essentially an element-wise multiplication and modulo operation.

Figure 12 illustrates the interplay among the vector unit, NTTU, and SPM during the NTT computation. From generating the input of NTT to using the output of NTT, the following steps are involved:

1. The vector unit generates the input for NTT and writes it to SPM.
2. NTTU reads from SPM, computes $\boldsymbol{W}_{N_1}\boldsymbol{X}$, and writes the result back to SPM.
3. The vector unit reads from SPM, performs modulo operations, computes $\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right) \odot \boldsymbol{W}_{N,N_1\times N_2}$, and writes the result back to SPM.
4. NTTU reads from SPM, computes $\left(\left(\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right) \odot \boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}\right)^T$, and writes the result back to SPM.
5. The vector unit reads from SPM, performs modulo operations and conducts subsequent computations.

These steps can be fully pipelined. Taking the first INTT (Line 2 in Algorithm 14) in `KeySwitch` (Algorithm 13) as an example, the following steps are

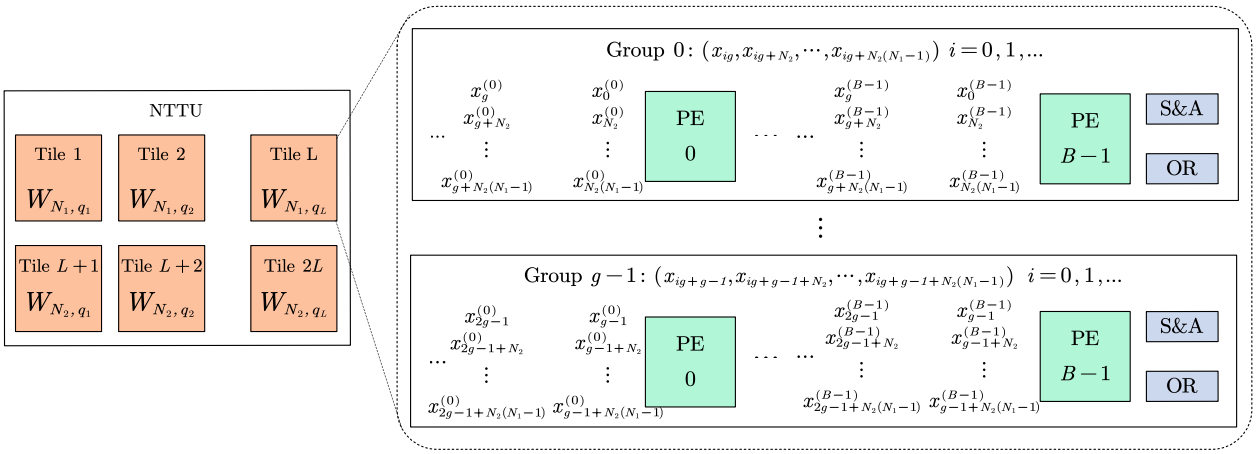


Fig. 11 Hierarchical organization of NTTU (using the same notation as in Fig. 5)

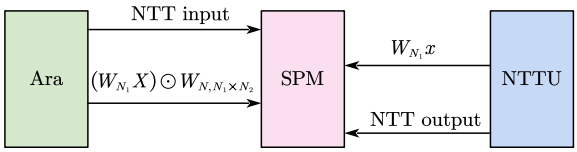


Fig. 12 Interplay of the vector unit, NTT Unit, and SPM in NTT computation

involved from computing the input of INTT (`DComp` in Algorithm 13) to using its output (`BConv`):

- Step 0: The vector unit computes `VecMulMontMod` (`DComp`) and writes the result back to SPM.
- Step 1: NTTU computes $\boldsymbol{W}_{N_1}\boldsymbol{X}$ and writes the result back to SPM.
- Step 2: The vector unit reads from SPM, performs modulo operations on the result of Step 1 (`VecNttMontMod`), computes $\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right)\odot\boldsymbol{W}_{N,N_1\times N_2}$ (`VecMulMontMod`), and writes the result back to SPM.
- Step 3: NTTU computes $\left(\left(\left(\boldsymbol{W}_{N_1}\boldsymbol{X}\right)\odot\boldsymbol{W}_{N,N_1\times N_2}\right)\boldsymbol{W}_{N_2}\right)^T$ and writes the result back to SPM.
- Step 4: The vector unit reads from SPM, performs modulo operations on the result of Step 3 (`VecNttMontMod`), and computes `VecMulMontMod` (in Algorithm 14).

The computation order of the vector unit in Steps 0, 2, and 4 is shown in Fig. 13. Each chunk, which is the input of one group in the NTTU, consists of all elements in the same column of $\boldsymbol{X}$, i.e., elements with stride N_2 in the same limb. The vector unit first computes different chunks within the same limb and then proceeds to the next limb. The NTTU performs Step 1 simultaneously. Whenever the vector unit completes Step 0 for a chunk and writes the result back to SPM, the NTTU reads from SPM, performs matrix–vector multiplication on this chunk, and writes the result back to SPM. When the vector unit completes Step 0 for all limbs, it can read from SPM and perform Step 2.

As mentioned in "Using decomposition method to compute NTT", Step 3 can only start after Step 2 finishes computing one complete limb rather than a chunk. If there is only one limb ($\ell = 1$), the vector unit needs to stall for one crossbar cycle to wait for outputs of the NTTU, as shown in Fig. 14b. If $\ell > 1$, the vector unit can immediately perform Step 4 without any stall, as shown in Fig. 14a.

The clock cycles of the NTTU and the vector unit differ, with the clock cycle of NTTU being approximately 40 times that of the vector unit. Section "Number of function units" will provide a method to select g (the number of groups within a tile) to match the running rates of the two, ensuring that the vector unit will not be idle.

Data layout

The pipeline described in Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline" requires the vector unit to perform computations on a chunk, which consists of elements with stride N_2 within a limb. This necessitates the use of constant-stride vector memory access instructions in Ara, which are significantly less efficient than unit-stride memory access instructions due to their involvement with multiple cache lines.

To circumvent the use of constant-stride memory access instructions, LP-HENN stores elements within a chunk, i.e., elements with stride N_2, contiguously. For a given vector index i, the corresponding memory address offset is calculated as $(i \bmod N_2)N_1 + \lfloor \frac{i}{N_2} \rfloor$. Figure 15 illustrates the mapping between indices in vector and physical memory addresses before and after the optimization. So the chunk mentioned in Sect. "Chunk&fusion optimization" is contiguous in storage rather than

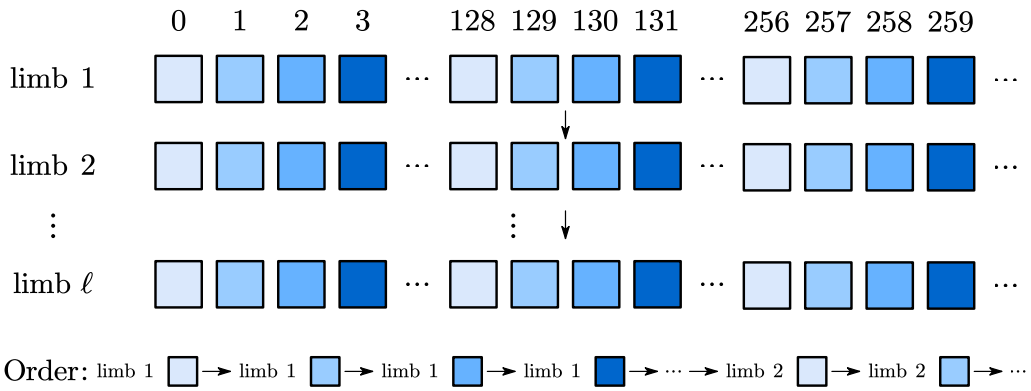


Fig. 13 Computing order of the vector unit. Each grid represents an element of a limb. Grids of the same color are within the same chunk

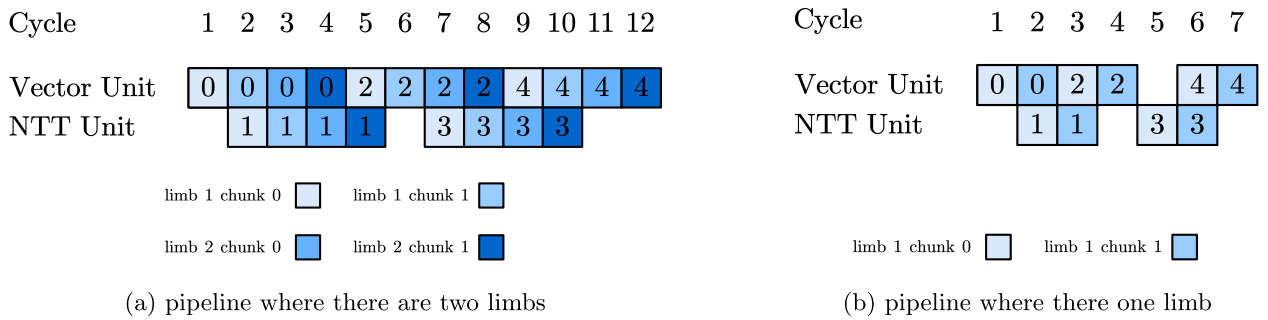


Fig. 14 Illustration of NTT pipeline in LP-HENN. For simplicity, this figure assumes there are 2 chunks in each limb. Each grid represents a chunk. Numbers in the grids denote the step, e.g., 0 denotes the vector unit is working on Step 0. The cycle is based on the crossbar cycle, not the cycle of the vector unit

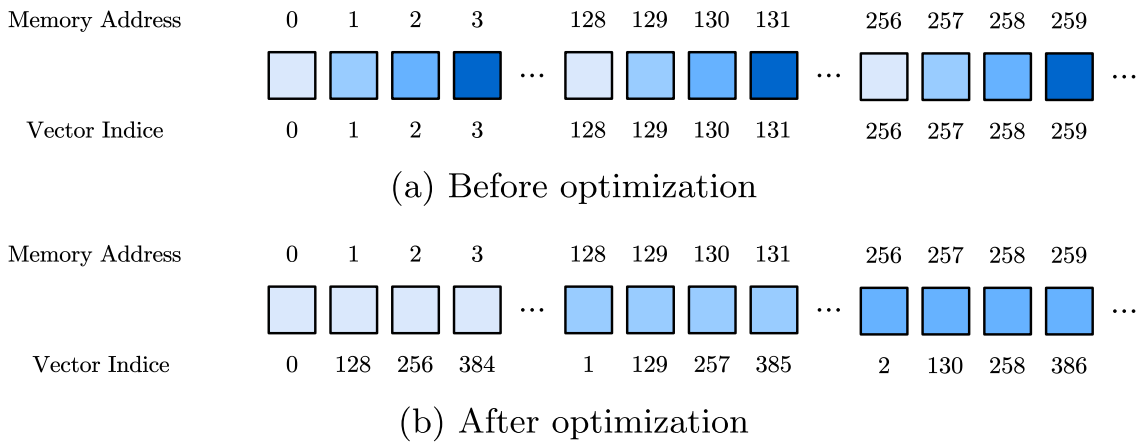


Fig. 15 Mapping between memory addresses and vector indices before and after optimization

comprising logically contiguous vector elements. This memory mapping is implemented at the software level, eliminating the need for additional hardware logic. With this optimization, accessing a chunk no longer necessitates the use of strided memory access instructions.

This optimization enhances the memory access efficiency of the vector unit in NTT computation without incurring additional overhead. As discussed in Sect. "Overview", all vector computations, except for NTT and automorphism, are element-wise operations. For element-wise vector operations, altering the order of element computation does not affect the result. In the case of automorphism, only $\boldsymbol{\nu}_2$ in Algorithm 5 requires adjustment. For instance, with $N = 2^{14}$ and $r = 1$, $\boldsymbol{\nu_2} = (0, 3277, 6554, 9831, 13108, \cdots)$. Upon applying this optimization, $\boldsymbol{\nu_2} = (0, 9881, 3379, 13260, 6758, \cdots)$.

Automatic parameter selection model

Overview

The design of FHE accelerators involves numerous software and hardware parameters, which collectively constitute a large parameter space. The software parameters are the parameters of the CKKS scheme, including the number of polynomial terms (N), the initial level of ciphertexts (L), the decomposition number (`dnum`) in `KeySwitch`, and the bit length (B) of the small primes (q_j). The hardware parameters include the capacity of SPM, the capacity of DRAM, and the number of function units, which are the number of groups in a tile (g) for the NTT unit and the number of lanes for the vector unit (l). The values of these parameters and their combinations profoundly influence the performance and energy efficiency of FHE accelerators.

To efficiently explore the optimal parameter combination within the design space, we develop a parameter selection model. The model serves the following purposes:

- For a given neural network application, it evaluates the constraints on the application's software and hardware parameters, including the minimum capacities of DRAM and SPM, and the minimum value of L.
- It evaluates the architecture's performance and power consumption under a given parameter set.
- It automatically selects the optimal parameter combination for LP-HENN considering power consumption and performance for a given application, using the Energy-Delay Product (EDP) as the evaluation metric.

The parameter selection model offers significant advantages in two key aspects. First, it automatically determines both cryptographic and architectural parameters based on application-specific characteristics, optimizing performance and energy efficiency without requiring manual parameter tuning. Second, the model accommodates user-specified constraints, such as upper bounds on DRAM capacity, on-chip buffer capacity, and power budget, subsequently identifying parameters that maximize performance and energy efficiency within these constraints.

The workflow of the parameter selection model is depicted in Fig. 16. First, optimal cryptographic parameters L, N, and `dnum` are determined, as described in Sect. "Cryptographic parameters". Subsequently, the number of function units l and g are established, with the process detailed in Sect. "Cryptographic parameters". The model then calculates the minimum capacity requirements for SPM and DRAM, as elaborated in Sect. "SPM capacity" and Sect. "DRAM capacity", respectively. Section "Number of function units" presents the evaluation methodology for assessing performance and power consumption of a given application under specific software/hardware parameter configurations.

Evaluation method of performance and power consumption

Performance is evaluated by counting the total number of addition and multiplication operations, as well as the total data transfers between the VRF and L1 cache, L1 cache and SPM, and SPM and DRAM for each primitive operation. The latency of a primitive operation is calculated as:

$$\max(\text{computation time}, \text{memory access time})$$

where:

$$\text{computation time} = \frac{\text{total number of additions and multiplications}}{\text{Ara's peak operations per second}}$$

and:

$$\text{memory access time} = \frac{\text{total data exchange volume between VRF and cache}}{\text{Ara's memory bandwidth}} + \frac{\text{SPM read/write volume}}{\text{SPM bandwidth}} + \frac{\text{DRAM read/write volumn}}{\text{DRAM bandwidth}}$$

The following components consume energy: Ara and the CVA6 core, SPM, DRAM, and ReRAM crossbars. The energy consumption of Ara and the CVA6 core is directly evaluated using the values reported in the literature (Zaruba and Benini 2019; Perotti et al. 2022). For SPM and DRAM, CACTI 6.5 (Muralimanohar et al. 2009) is employed for power consumption evaluation. The model monitors the number of accesses to SPM and DRAM, which is calculated as data read/write volume divided by the block size (1024 B, the width of the SPM port). ReRAM power consumption is calculated by multiplying the power consumption of a single crossbar operation by the total number of operations across all crossbars. The model keeps track of the total number of crossbar operations, and the power consumption of a single crossbar operation is based on RM-NTT (Park et al. 2022).

Cryptographic parameters

The cryptographic parameters include N, L, B, and `dnum`. As discussed in Sect. "Basic vector operations and

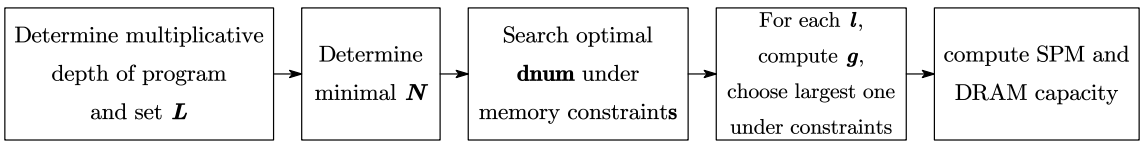


Fig. 16 Workflow of the parameter selection model

their implementations", B is fixed at 30 bits. However, N, L, and dnum still constitute a vast parameter space. The parameter selection process reduces this search space by considering both the impact of cryptographic parameters on FHE application performance and the constraints among these parameters. Figure 17 illustrates these relationships and constraints.

Larger values of N and L increase both the computational cost of HE operations and the size of ciphertexts, plaintexts, and evaluation keys. Consequently, smaller values of N and L improve performance and energy efficiency. The parameter dnum influences the computational cost of HMult and HRot operations, though this relationship is not strictly monotonic—there exists an optimal dnum that minimizes computational cost. However, increasing dnum leads to larger evaluation keys, resulting in higher bandwidth requirements during HMult and HRot computations and increased DRAM capacity demands. Since computational costs have a more significant impact on energy efficiency compared to memory access, the dnum configuration that minimizes computational cost also yields optimal energy efficiency.

These parameters are subject to two primary constraints:

1. L must be greater than or equal to the multiplicative depth of the program plus one.
2. At a given security level (predominantly 128-bit for FHE applications), the bit length of $P \cdot Q$ (computed as $LB + \lceil \frac{LB}{\text{dnum}} \rceil$) has an upper bound for any given N. For instance, at the 128-bit security level, the maximum bit length of $P \cdot Q$ is 111 bits when $N = 4096$ and 220 bits when $N = 8192$.

Based on these impacts and constraints, the parameter selection process employs heuristics to determine cryptographic parameters as follows:

1. First, L is determined by the program's multiplicative depth, setting L equal to the multiplicative depth plus one, thus identifying the minimum L that satisfies Constraint 1.
2. Next, the minimum N satisfying Constraint 2 is determined. The lower bound for N is the smallest value that ensures the bit length of $P \cdot Q$ is at least $B(L+1)$ (where BL represents the bit length of Q, with B being Q's minimum bit width when $\text{dnum} = L$). However, when DRAM capacity is constrained, N may need to be larger than this lower bound, as larger N values might allow for smaller dnum values that satisfy DRAM capacity constraints.
3. Finally, the optimal dnum is selected. While dnum typically ranges over integers in $[1, L]$, DRAM capacity constraints may lower this upper bound. The optimal dnum is determined by iterating through possible values and selecting the one that minimizes computational cost.

This parameter selection process significantly simplifies the parameter space exploration by requiring only the iteration over possible dnum values.

Number of function units

This section presents the calculation method for the number of function units, specifically the lane number of the vector unit (l) and the number of groups in a tile (g) for the NTT unit. We first present the method for calculating g given a specific lane number of the vector unit, followed by the approach for determining the lane number of the vector unit.

Section "Hierarchical organization of NTTU" describes the hierarchical structure of the NTTU, which consists of $2L$ tiles, with L tiles allocated for $\boldsymbol{W}_{N_1}$ and L tiles for $\boldsymbol{W}_{N_2}$. Each tile comprises g identical groups, and each group

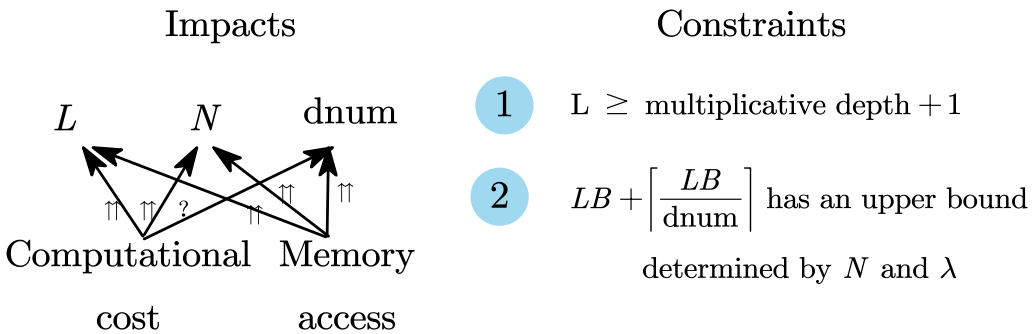


Fig. 17 Impacts of cryptographic parameters on computational cost and memory access of FHE applications and their inter-parameter constraints. ↑↑ indicates a positive correlation, while "?" denotes an existing but non-monotonic relationship

Table 6 Number of ciphertexts simultaneously cached at each layer of LoLa neural networks

Layer	Number of Ciphertexts to be Cached
Convolution layer (with convolution representation)	4
Convolution layer (with dense representation)	5
Dense layer (with stacked representation)	4
Activation layer	3

Table 7 Configuration for evaluation of LP-HENN

Parameter	Value
L	7
N	8192 (MNIST), 16384 (CIFAR, HELR)
dnum	7 (MNIST), 4 (CIFAR), 3 (HELR)
B	30
Number of lane	16
Crossbar size	128Ã-128
Total crossbar number	16800 (MNIST), 20160 (CIFAR, HELR)
SPM capacity	4.375 MB
DRAM capacity	2.08 GB

contains B identical PEs. The crossbars within each PE store either $\boldsymbol{W}_{N_1}$ or $\boldsymbol{W}_{N_2}$. As illustrated in Sect. "Using array of crossbars to compute NTT", a single cell of a crossbar stores only 1 bit. Assuming the size of a crossbar is c, the number of crossbars within a PE is given by $\lceil \frac{N_1}{\lfloor \frac{c}{B} \rfloor} \rceil$ for $\boldsymbol{W}_{N_1}$ or $\lceil \frac{N_2}{\lfloor \frac{c}{B} \rfloor} \rceil$ for $\boldsymbol{W}_{N_2}$. For example, when $N = 16384$, $N_1 = N_2 = 128$, and $B = 30$, the count of 128Ã-128 crossbars within a PE is 32.

To ensure the pipeline described in Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline" works properly and avoid vector unit idling, the rate at which crossbars generate chunks should be greater than or equal to the rate at which Ara processes chunks. Therefore, g is given by:

$$g = \frac{\text{crossbar cycle} \times \text{lane number} \times \text{function units per lane}}{\text{Ara cycle} \times \text{count of multiplication and addition per element} \times N_1 \text{ or } N_2}$$

Table 8 Performance and energy consumption comparison among LP-HENN, FxHENN, F1, and CPU implementations across benchmarks

Accelerator	MNIST		CIFAR		HELR	
	Latency (s)	Energy (J)	Latency (s)	Energy (J)	Latency (s)	Energy (J)
CPU	2.2	1936.00	730	642400.00	–	–
F1	0.00017	0.031	0.241	43.48	0.00115	0.21
FxHENN	0.19	1.90	54.1	541.00	–	–
LP-HENN	0.034	0.08	22.938	53.89	0.378	0.73

Table 9 Proportion of Computation and Memory Access of (I)NTT in LP-HENN for LoLa-MNIST and LoLa-CIFAR

	NTT computation	NTT memory access
LoLa-MNIST	56.8%	38.5%
LoLa-CIFAR	52.8%	40.9%
HELR	48.1%	51.0%

Table 10 Data movement (DM) and bandwidth (BW) between SPM, VRF, and ReRAM across benchmarks

Benchmark	VFR-SPM		DRAM-SPM		ReRAM-SPM	
	DM (GB)	BW (GB/s)	DM (GB)	BW (GB/s)	DM (GB)	BW (GB/s)
LoLa-MNIST	1.28	37.57	0.17	4.86	0.33	9.64
LoLa-CIFAR	802.78	35.00	92.18	4.02	218.83	9.54
HELR	7.36	19.46	3.38	8.95	2.50	6.61

Function units per lane is 2 according to Sect. "Vector unit: structure and workflow". For tiles storing $\boldsymbol{W}_{N_1}$, the rate at which the NTTU produces $\boldsymbol{W}_{N_1}x$ outputs should be greater than or equal to the rate at which the vector unit computes Stage 2 in Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline" (one `VecNttMontMod` and one `VecMulMontMod`). This ensures that the vector unit can compute Stage 2 continuously without stalling "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline".

For tiles storing $\boldsymbol{W}_{N_2}$, the rate at which the NTTU generates $x\boldsymbol{W}_{N_2}$ outputs should be greater than or equal to the rate at which the vector unit subsequently uses the NTT results (Stage 4 in Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline"). This ensures that Ara can compute Stage 4 without stalling upon the completion of Stage 2. The worst-case scenario arises in the NTT of `ModDown` in HRotate, wherein the vector unit only needs to compute `VecNttMontMod` after NTT, resulting in g taking the maximum value.

In the absence of power budget constraints, maximum performance (minimum latency) is achieved when the number of vector lanes l reaches its maximum value (16). Experimental results indicate that varying the number of vector lanes has minimal impact on energy consumption. From an EDP perspective, $l = 16$ yields optimal results. However, when power budget constraints are present, power consumption decreases with smaller values of l. In such cases, the optimal strategy is to select the largest value of l that satisfies the power constraint.

SPM capacity

The SPM serves as a cache to minimize access to DRAM. It does not cache data that is accessed only once, as this would not reduce DRAM accesses. In LoLa-MNIST and LoLa-CIFAR, the weights of neural networks are stored in plaintext. Since all plaintexts are used only once, the SPM does not cache them. Similarly, the SPM does not cache evaluation keys. In LoLa-MNIST and LoLa-CIFAR, the number of HMult operations ranges between 1 and 2, respectively, making the benefit of caching the evaluation key for HMult negligible. For $\mathbf{evk}_{\text{rot}}^{(r)}$, since HRotate operations with the same shift are separated by dozens of HRotate operations with different shifts, caching multiple $\mathbf{evk}_{\text{rot}}^{(r)}$ would require a large SPM capacity, which is not cost-effective. Consequently, LP-HENN reads the evaluation keys from DRAM in each HRotate and HMult operation. Experimental results demonstrate that the `KeySwitch` operation is compute-bound rather than memory-bound, and reading evaluation keys from DRAM has no significant impact on performance.

Therefore, the SPM only caches ciphertexts, and its capacity should be sufficient to ensure that there are no read-after-write scenarios for ciphertexts. Analyzing the different types of neural network layers in LoLa, the number of ciphertexts that need to be cached simultaneously at any given time for each layer is presented in Table 6.

Consequently, considering the tradeoff between cost and performance, the minimum SPM capacity should be able to accommodate 5 ciphertexts.

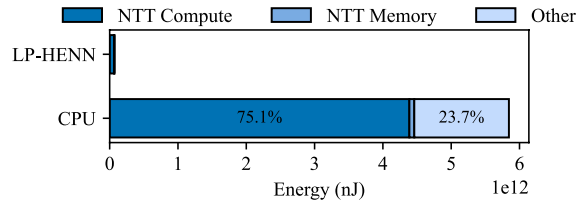


Fig. 18 Energy consumption breakdown of CPU and LP-HENN on LoLa-CIFAR

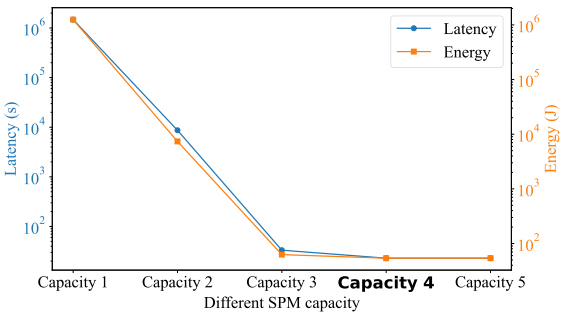


Fig. 19 Impact of SPM capacity on LoLa-CIFAR's latency and energy consumption

DRAM capacity

The following data is stored in DRAM:

- Input ciphertexts
- Plaintexts storing weights
- Evaluation keys
- Pre-computed constants, such as $\hat{q}_j^{-1}$, $[\hat{q}_j]_{p_i}$ in BConv, and $\boldsymbol{W}_{N,N_1\times N_2}$ in NTT.

For input ciphertexts, only the number of ciphertexts for the first layer of the neural network needs to be considered, as the number of ciphertexts decreases throughout the computation process of LoLa.

The model counts the number of plaintexts used for storing weights in the neural network. For plaintexts in sparse format (Brutzkus et al. 2019), where all elements of the vector are identical, storing the entire polynomial is unnecessary. After encoding, sparse format vectors yield constant polynomials with only the constant term being non-zero. Therefore, only the constant term needs to be stored, and this optimization significantly reduces the DRAM storage requirements.

Regarding evaluation keys, as discussed in Sect. 2.1, all HMult operations require only one $\mathbf{evk}_{\text{Mult}}$, while each HRotate operation requires a distinct $\mathbf{evk}_{\text{rot}}^{(r)}$ for each unique shift. The model counts the number of different shifts used in the neural network application.

Counting the precise number of constants is unnecessary as their size is negligible compared to the size of a single ciphertext.

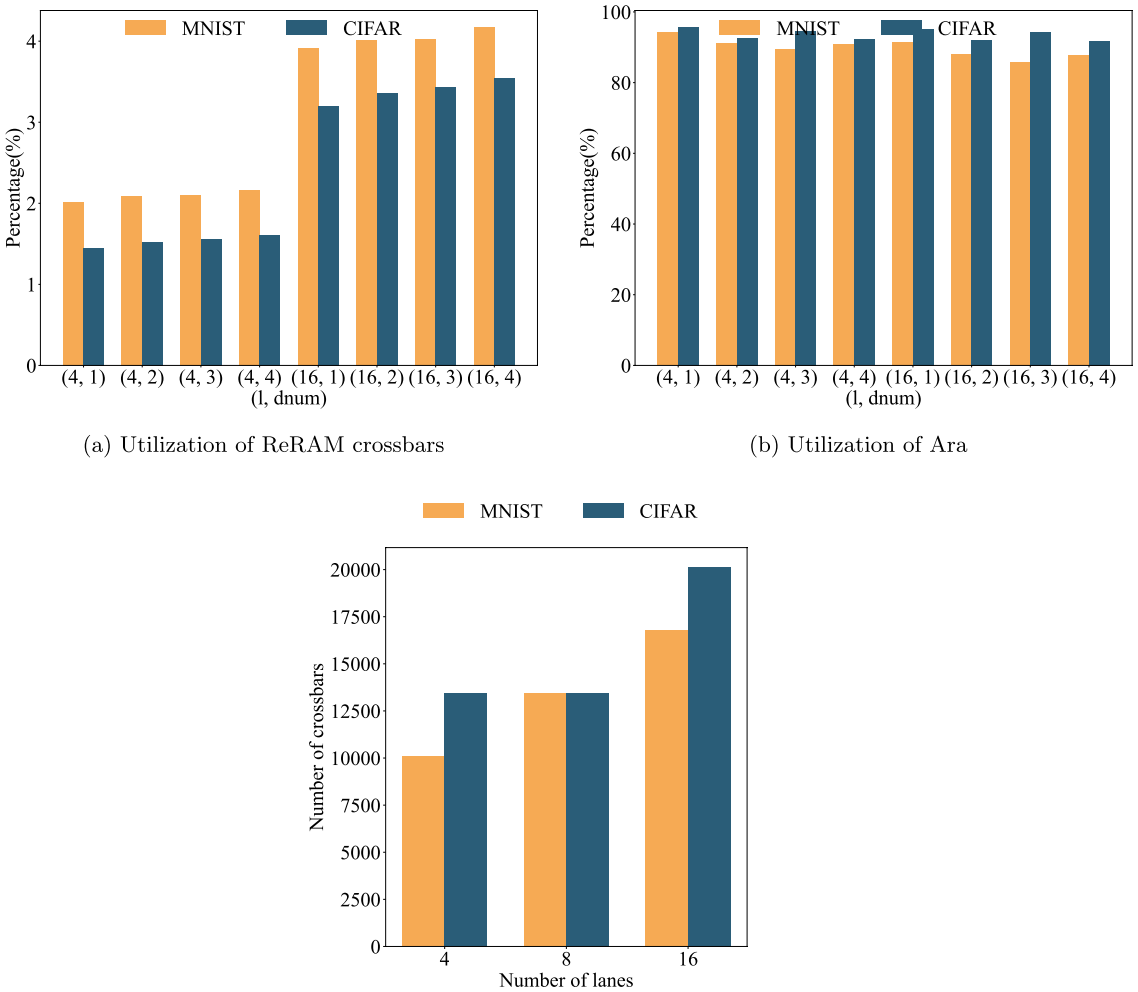
Results

Experimental setup

LP-HENN was evaluated using two homomorphic encryption convolutional neural network (HE-CNN) inference applications—LoLa-MNIST and LoLa-CIFAR (Brutzkus et al. 2019)—along with homomorphic encryption logistic regression (HELR) training (Han et al. 2019). LoLa-MNIST is a 5-layer HE-CNN designed for classifying the MNIST dataset (LeCun et al. 2010), achieving an accuracy of 98.95%. LoLa-CIFAR is a 5-layer HE-CNN used for classifying the CIFAR-10 dataset (Krizhevsky et al. 2009), achieving an accuracy of 74.1%. LoLa-CIFAR is significantly larger than LoLa-MNIST, as its input has 3072 (32 Ã- 32 Ã- 3) nodes, compared to the 784 (28 Ã- 28) input nodes of LoLa-MNIST. Both networks conduct inference on a single image at a time. HELR compute a single batch of logistic regression training with up to 256 features, and 256 samples per batch.

Table Results" presents the optimal parameter configuration for LP-HENN, evaluated using the model described in Sect. "Automatic parameter selection model", with EDP as the evaluation metric.

We use FxHENN (Zhu et al. 2023), F1(Samardzic et al. 2021) and a CPU implementation (Brutzkus et al. 2019) as baselines. FxHENN is an FPGA accelerator specifically designed for HE-CNN inference in edge and embedded computing environments, boasting high energy efficiency. It uses the ALINX ACU15EG (equipped with the Xilinx Zynq UltraScale+ MPSoC XCZU15EG device), which is a high-end embedded FPGA platform with 3528 DSP slices, 26.2 Mbit on-chip BRAM, and 31.5 Mbit on-chip URAM. F1 is a server-oriented, state-of-the-art ASIC FHE accelerator that supports polynomial degrees (N) ranging from 1024 to 16384, matching LP-HENN's capabilities, and targets BGV, CKKS, and GSW schemes. More recent ASIC FHE accelerators such as ARK(Kim et al. 2022a) and SHARP(Kim et al. 2023) were not selected for comparison as they primarily focus on deep FHE workloads requiring bootstrapping operations, with N fixed at 65536. The scalability analysis for deep workloads will be discussed in Sect. "Performance bottlenecks and scalability limitations". The CPU implementation is based on SEAL 2.3 (Chen et al. 2017), and the reference machine is an Azure standard B8ms virtual machine with 8 vCPUs and 32 GB of RAM.



(a) Utilization of ReRAM crossbars

(b) Utilization of Ara

(c) Number of crossbars v.s. number of lanes

Fig. 20 Hardware utilization of ReRAM Crossbars and Ara under various parameter sets with supporting results

Performance and energy consumption

Table 8" presents the performance and energy consumption of LP-HENN running benchmarks under optimal parameters. Compared to CPU, LP-HENN achieves a speedup of 64.54Ã- and 31.82Ã- on LoLa-MNIST and LoLa-CIFAR, respectively. Furthermore, LP-HENN outperforms FxHENN by 5.57Ã- and 2.36Ã- on LoLa-MNIST and LoLa-CIFAR, respectively.

In terms of energy efficiency, LP-HENN demonstrates 23944.88Ã- and 11920.56Ã- improvements compared to CPU on LoLa-MNIST and LoLa-CIFAR, respectively. Moreover, LP-HENN surpasses FxHENN by 23.5Ã- and 10.04Ã- in energy efficiency on LoLa-MNIST and LoLa-CIFAR, respectively.

Compared to F1, LP-HENN exhibits significantly higher latency, while F1 achieves speedups of several thousand to tens of thousands over CPU implementations. Regarding energy efficiency, F1 demonstrates lower energy consumption than LP-HENN, with differences ranging from 1.2 to several-fold. In HELR evaluations, the energy consumption difference between the two approaches reaches tens of times; however, this disparity likely stems from evaluation methodology variations rather than actual energy consumption differences. LP-HENN's higher energy consumption is primarily attributed to its extended execution time and older manufacturing process (28 nm) compared to F1's more advanced 14/12 nm process.

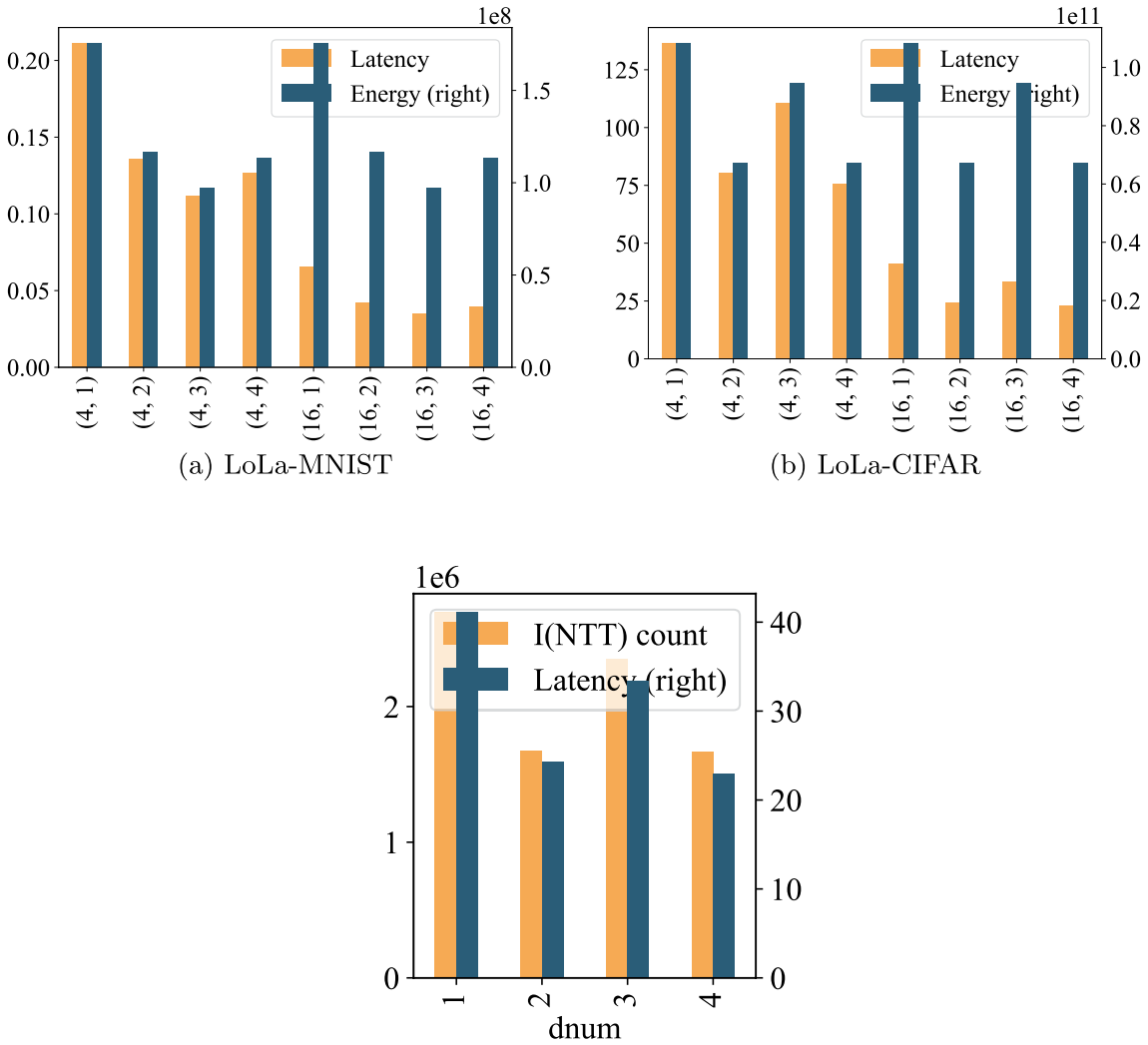


Fig. 21 Latency and energy consumption under different parameter sets and supporting result

Nevertheless, F1's power consumption of 180.4 W exceeds battery-powered operational ranges, making it unsuitable for edge computing scenarios. In contrast, LP-HENN maintains power consumption between 1.93 W and 2.35 W across various benchmarks, with the potential for further reduction through power budget constraints in the automatic parameter selection model-albeit at the cost of increased latency. Consequently, LP-HENN proves more suitable than F1 for edge computing applications.

Several factors contribute to the performance improvements. First, expressing FHE operations using vector assembly instructions significantly reduces redundant computation, resulting in tens of times reduction in computation compared to CPU. Although LoLa, implemented using SEAL, utilizes OpenMP for parallelization, an in-depth analysis reveals that the majority of the execution time is consumed by the overhead associated with OpenMP itself. Second, the vector processor features a higher number of computing units compared to the CPU and operates at a higher clock frequency than FxHENN. Third, as highlighted in Sect. "Energy and performance implications of NTT in FHE acceleration", NTT is the primary bottleneck for both performance and energy consumption. LP-HENN offloads a large portion of NTT computation and memory access to the NTTU, and through the pipeline optimization described in Sect. "Collaborative computation process of NTT with

NTTU and Vector Unit and pipeline", the processing time of the NTTU can be effectively overlapped. Table 9 presents the proportion of (I)NTT computation and memory access in LP-HENN on LoLa-MNIST and LoLa-CIFAR. Compared to Table 5, on LoLa-MNIST, the proportion of computation in LE-HENN decreases by 20.9%, and the memory access decreases by 50.6%; on LoLa-CIFAR, the proportion of computation in LE-HENN decreases by 23.3%, and the memory access decreases by 49.8%. These results demonstrate the significant role of NTTU in reducing the computation and memory access overhead of NTT.

The reduction in energy consumption primarily comes from the decrease in computation and high energy efficiency of the vector processor and ReRAM crossbars. Figure 18 presents a breakdown of the energy consumption of CPU and LP-HENN under the LoLa-CIFAR application. To ensure a fair comparison, the computation on the CPU is set to be identical to those of LP-HENN, utilizing the results from the model, while NTT employs the Cooley-Tukey algorithm. The computation and memory access energy of NTT is substantially reduced, and the energy consumption of LP-HENN is almost concentrated on the computation of NTT. This is because using the matrix–vector multiplication method significantly increases the computational overhead of NTT compared to the Cooley-Tukey algorithm. Despite the increase in computation, it is easily vectorizable and has a very high degree of parallelism, greatly reducing data dependencies. When combined with pipelining techniques, the latency of this approach on vector processors is significantly lower than that of the Cooley-Tukey algorithm.

Table 10 presents the data movement and memory bandwidth between SPM, VRF, ReRAM, and DRAM across several benchmarks. Despite significant variations in data movement across applications, the bandwidth remains relatively stable.

Impact of ReRAM on NTT Performance The implementation of ReRAM crossbars affects system performance in three significant aspects. First, it reduces the computational load on the vector unit. According to Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline", the vector unit no longer needs to compute the complete NTT, but instead performs only element-wise modular multiplication and modular reduction operations, resulting in an 64.9% reduction in computational cost. Second, it decreases memory access time for the vector unit. By utilizing matrix–vector multiplication, ReRAM crossbars eliminate the frequent memory accesses required by the Cooley-Tukey butterfly algorithm, leading to a 92.9% reduction in total memory access volume for NTT operations across both ReRAM crossbars and the vector unit. Third, it introduces additional overhead from ReRAM crossbars. The ADC and DAC components in ReRAM crossbars introduce extra latency during matrix–vector multiplication operations, with each operation requiring approximately 40 ns. However, as described in Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and pipeline" pipeline architecture, this latency can be effectively masked in most scenarios by properly balancing the number of lanes l with groups of ReRAM crossbars g (detailed in Sect. "Number of function units"). The latency becomes visible only in the specific case where the ciphertext consists of a single limb, causing Stage 2's ReRAM crossbar latency to remain unmasked. Nevertheless, this limitation has negligible impact on overall system performance.

Performance with varying hardware resources

VRF capacity Adjusting the VRF capacity, either upward or downward, does not significantly impact LP-HENN's performance. Changes in VRF size only affect the chunk size in the chunk&fusion optimization discussed in Sect. "Chunk&fusion optimization", without introducing additional memory access overhead.

DRAM capacity The DRAM capacity cannot be reduced below the specifications outlined in Sect. "DRAM capacity", as it must accommodate all input ciphertexts, input plaintexts, and evaluation keys required by the workload. A smaller DRAM capacity would be insufficient to store all inputs, while increasing the capacity would not yield performance benefits as the additional space would remain unutilized.

SPM capacity The SPM capacity significantly impacts both the performance and energy consumption of FHE workloads. Figure 19 presents an analysis of latency and energy consumption for LoLa-CIFAR across five SPM capacity configurations: (1) no SPM, where ReRAM crossbars communicate with the vector unit through DRAM; (2) SPM capacity sufficient for two limbs, enabling ReRAM crossbar-vector unit interaction via SPM, but insufficient for a complete ciphertext; (3) SPM capacity adequate for one ciphertext but insufficient for caching ciphertexts across primitive operations; (4) the capacity specified in Sect. "SPM capacity" and implemented in LP-HENN, allowing ciphertext caching across primitive operations but not evaluation keys; and (5) SPM capacity sufficient to cache all evaluation keys. Cases 1 and 2 exhibit substantially higher latency and lower energy efficiency due to limited DRAM bandwidth, while the crossbar latency cannot be masked during NTT computation as detailed in Sect. "Collaborative computation process of NTT with NTTU and Vector Unit and

pipeline". Case 3, with an SPM capacity one-fifth that of Case 4, requires DRAM access for reading and writing operands and outputs during each primitive operation, resulting in a 44.97% increase in latency and a 16.2% increase in energy consumption. This performance trade-off remains acceptable when SPM budget is constrained. Case 4, compared to Case 5, introduces additional DRAM reads for evaluation keys during HMult and HRot operations. However, due to the relatively low frequency of these operations, the performance impact is negligible, validating the effectiveness of the approach described in Sect. "SPM capacity".

Hardware utilization under different parameter sets

The utilization of ReRAM and Ara under various parameter combinations (l, `dnum`) is illustrated in Figs. "20a and 20b, respectively. Ara's utilization ranges between 85.9 and 95.6%, while ReRAM's utilization is extremely low, ranging from 1.4 to 4.2%. The significantly lower utilization of ReRAM crossbars compared to Ara can be attributed to two primary factors. First, Ara is always kept busy, while ReRAM crossbars are only used during NTT computation. Second, to minimize the latency of NTT, crossbars are replicated multiple times (Sect. "Hierarchical organization of NTTU ").

As the number of lanes increases from 4 to 16, the utilization of ReRAM crossbars doubles. This can be explained using Fig. 20c, which shows the variation in the number of crossbars with respect to the number of lanes. When the number of lanes is 16, the number of crossbars is nearly twice that of when the number of lanes is 4. Therefore, the number of NTT computations per unit time is roughly 4 times higher, and the number of crossbars is approximately 2 times higher, resulting in the utilization of crossbars being nearly doubled.

Sensitivity study: performance and energy consumption under different parameter sets

Figures 21a and b illustrate the latency and energy consumption of LP-HENN under various parameter combinations on the LoLa-MNIST and LoLa-CIFAR, respectively. The parameters provided in Table 7 yield the minimum latency while exhibiting nearly the lowest energy consumption, resulting in the optimal EDP. Increasing the number of lanes from 4 to 16 significantly reduces latency with a slight increase in energy consumption, making 16 lanes clearly superior to 4 in terms of EDP. The graphs reveal that `dnum` substantially influences both latency and energy consumption. Figure 21c presents the number of (I)NTT operations and total latency of LoLa-CIFAR with 16 lanes and varying `dnum` to explain this phenomenon. According to Fig. 18, ReRAM is the primary energy consumer, and its consumption is directly proportional to the number of (I)NTT operations. Consequently, the total energy consumption is approximately proportional to the number of (I)NTT operations. The variations in energy consumption and latency depicted in Fig. 21b for 16 lanes and `dnum` values of 1, 2, 3, and 4 are consistent with the trend of (I)NTT operation count presented in Fig. 21c, demonstrating that `dnum` influences energy consumption and latency by impacting the number of (I)NTT operations.

Discussion

Performance bottlenecks and scalability limitations

LP-HENN primarily targets FHE workloads with low multiplicative depth, showing notable limitations for deeper FHE computations. These limitations stem from two main factors: the performance bottleneck of the vector processor and the scaling challenges in the number of crossbars in NTTU as multiplicative depth increases.

From a performance perspective, LP-HENN's computational capabilities are constrained by its vector processor, Ara. Even at maximum configuration (16 lanes), Ara achieves a peak performance of only 33 DP-GFLOPS, significantly lower than specialized ASIC FHE accelerators (such as F1) or GPU accelerators. This performance bottleneck cannot be addressed by simply increasing the number of crossbars in NTTU, as the fundamental limitation lies in the vector processor's computational capacity.

Regarding architectural scalability, the number of crossbars in LP-HENN exhibits a linear relationship with the cryptographic parameter L. Consequently, supporting FHE programs with increased multiplicative depth requires proportionally more crossbars. This limitation becomes particularly pronounced when supporting unlimited-depth multiplication operations. Such operations necessitate Bootstrapping to refresh ciphertext levels. However, Bootstrapping not only incurs substantial computational overhead (involving hundreds of HMult and HRotate operations) but also demands larger cryptographic parameters: N must be at least 32768, and L must exceed twenty. These requirements would necessitate a several-fold increase in the required number of crossbars compared to the current implementation.

Security

This section analyzes the security properties of the LP-HENN architecture from two perspectives: the inherent security guarantees of FHE and protection against side-channel attacks.

From the perspective of FHE security, LP-HENN inherits the security properties of FHE since it is purely a hardware implementation without modifications to the underlying algorithms. The architecture maintains security as long as the application's parameter selection meets the security level λ requirements. A key security feature of FHE is that data remains encrypted throughout processing—even if data leakage occurs during computation, only the encrypted form is exposed, preserving the confidentiality of the original information.

Regarding side-channel attack prevention, LP-HENN exhibits inherent advantages due to a fundamental property of FHE programs: their execution paths are static and free from data-dependent branching. Consequently, memory access patterns remain independent of input data, resulting in stable bandwidth utilization patterns that do not fluctuate with varying inputs. This bandwidth stability effectively mitigates side-channel attacks based on bandwidth analysis. Since bandwidth usage patterns remain constant regardless of input data, attackers cannot infer any information about the input by monitoring bandwidth variations, thus fundamentally eliminating the possibility of such side-channel attacks.

Mitigation of ReRAM variability

As discussed in Sect. "Using array of crossbars to compute NTT", during RRAM programming, cell conductance may deviate from target values due to the stochastic switching behavior of RRAM, potentially leading to computational errors. While conventional VMM operations in neural networks can tolerate some variation, NTT operations require stricter conductance control to maintain accuracy.

The error probability in ReRAM crossbars is primarily determined by two device characteristics: the ON/OFF ratio and device variation. A higher ON/OFF ratio and lower device variation correspond to reduced error probability. Research by (Park et al. 2022) demonstrates that when device variation remains within 0.005 and the ON/OFF ratio exceeds 200, the error probability approaches zero with optimal ADC quantization levels. Therefore, appropriate device selection and ADC quantization can effectively mitigate these errors. As mentioned in Sect. "Using array of crossbars to compute NTT", to minimize error probability, we opted for smaller crossbar arrays (128Ã-128) and single-bit storage per cell.

An additional method for reducing error probability in devices with higher variation involves partial activation of crossbar rows. By setting inactive rows to zero, we can decrease the bitwidth of accumulated sums. For instance, with device variation within 0.1, activating only 32 rows simultaneously still maintains zero error probability. This operational mode is readily compatible with LP-HENN using either single-step or two-step recursive Cooley-Tukey algorithms.

Conclusion

This paper proposed LP-HENN, a novel low-power, energy-efficient FHE accelerator architecture targeting power-constrained application scenarios, such as edge devices. Given the static nature of FHE workloads and the fact that FHE operations process long vectors, employing a vector processor is a natural choice due to its high energy efficiency. We present how FHE primitive operations can be decomposed into basic vector operations and provide RISC-V vector instruction descriptions for these operations. Through the analysis of two HE-CNN applications, it is discovered that NTT is the primary bottleneck in terms of performance and power consumption. Since ReRAM crossbars are highly energy-efficient dot product engines, LP-HENN offloads NTT computation to the ReRAM crossbars. To reduce the number of ReRAM crossbars, we adopt a decomposition method to compute NTT, which introduces additional element-wise multiplication. As ReRAM crossbars cannot perform modulo and element-wise multiplication operations, we design a pipeline that enables collaboration between the vector processor and ReRAM crossbars, allowing the computation time of crossbars to be overlapped. The design of FHE accelerators involves numerous software and hardware parameters; therefore, we develop a parameter selection model to automatically determine the optimal parameter set that minimizes EDP. LP-HENN is evaluated using two HE-CNN applications, LoLa-MNIST and LoLa-CIFAR, and compared with a CPU implementation and FxHENN. For the LoLa-CIFAR application, LP-HENN achieves a 31.82Ã- and 11920.56Ã- improvement in performance and energy efficiency, respectively, compared to CPU. Compared to FxHENN, the state-of-the-art FHE accelerator with high energy efficiency for HE-CNNs targeting edge devices, LP-HENN achieves a 2.36Ã- and 10.04Ã- improvement in performance and energy efficiency, respectively.

Acknowledgements

Not applicable.

Authors' contributions

All the authors read and approved the final manuscript.

Funding

This research was supported by the National Key R&D Program of China (Grant No.2023YFB4503201) and the Strategic Priority Research Program of the Chinese Academy of Sciences (Grant No.XDB44030200).

Availability of data and materials

Data available within the article.

Declarations

Competing interests

The authors declare that they have no competing interests.

Received: 5 September 2024 Accepted: 8 January 2025
Published online: 30 May 2025

References

Agrawal R, Castro L, Yang G, Juvekar C, Yazicigil R, Chandrakasan A, Vaikuntanathan V, Joshi A (2023) Fab: an fpga-based accelerator for bootstrappable fully homomorphic encryption. In: 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 882–895. https://doi.org/10.1109/HPCA56546.2023.10070953

Al Badawi A, Bates J, Bergamaschi F, Cousins DB, Erabelli S, Genise, N, Halevi S, Hunt H, Kim A, Lee Y, et al (2022) Openfhe: Open-source fully homomorphic encryption library. In: Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography, pp. 53–63. https://doi.org/10.11591/ijeecs.v34.i3.pp1989-1998

Azad Z, Yang G, Agrawal R, Petrisko D, Taylor M, Joshi A (2023) Rise: Risc-v soc for en/decryption acceleration on the edge for homomorphic encryption. IEEE Transactions on Very Large Scale Integration (VLSI) Systems https://doi.org/10.1109/tvlsi.2023.3288754

Azad Z, Yang G, Agrawal R, Petrisko D, Taylor M, Joshi A (2022) Race: Risc-v soc for en/decryption acceleration on the edge for homomorphic computation. In: Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design, pp. 1–6. https://doi.org/10.1145/3531437.3539725

Badawi AA, Polyakov Y (2023) Demystifying bootstrapping in fully homomorphic encryption. Cryptology ePrint Archive, Paper 2023/149

Barrett P (1986) Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor. In: Conference on the Theory and Application of Cryptographic Techniques, pp. 311–323. https://doi.org/10.1007/3-540-47721-7_24

Brakerski Z, Gentry C, Vaikuntanathan V (2014) (leveled) fully homomorphic encryption without bootstrapping. ACM Trans. Computat. Theory (TOCT) 6(3):1–36. https://doi.org/10.1145/2090236.2090262

Brutzkus A, Gilad-Bachrach R, Elisha O (2019) Low latency privacy preserving inference. In: International Conference on Machine Learning, pp. 812–821. https://doi.org/10.1109/jiot.2020.3003468

Cavalcante M, Schuiki F, Zaruba F, Schaffner M, Benini L (2019) Ara: A 1-ghz+ scalable and energy-efficient risc-v vector processor with multiprecision floating-point support in 22-nm fd-soi. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 28(2), 530–543 https://doi.org/10.1109/tvlsi.2019.2950087

Chen H, Laine K, Player R (2017) Simple encrypted arithmetic library-seal v2. 1. In: Financial Cryptography and Data Security: FC 2017 International Workshops, WAHC, BITCOIN, VOTING, WTSC, and TA, Sliema, Malta, April 7, 2017, Revised Selected Papers 21, pp. 3–18. https://doi.org/10.1007/978-3-319-70278-0_1

Cheon JH, Kim A, Kim M, Song Y (2017) Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology–ASIACRYPT 2017: 23rd international conference on the theory and applications of cryptology and information security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23, pp. 409–437.https://doi.org/10.1007/978-3-319-70694-8_15

Cheon JH, Han K, Kim A, Kim M, Song Y (2019) A full rns variant of approximate homomorphic encryption. In: Selected Areas in Cryptography–SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25. SAC, pp. 347–368. https://doi.org/10.1007/978-3-030-10970-7_16

Chou T, Tang W, Botimer J, Zhang Z (2019) CASCADE: connecting RRAMs to extend analog dataflow in an end-to-end in-memory processing paradigm. In: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. MICRO '52, pp. 114–125, New York, NY, USA.https://doi.org/10.1145/3352460.3358328

Ducas L, Micciancio D (2015) Fhew: bootstrapping homomorphic encryption in less than a second. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 617–640. https://doi.org/10.1007/978-3-662-46800-5_24

Fan J, Vercauteren F (2012) Somewhat practical fully homomorphic encryption. Cryptol ePrint Arch. https://doi.org/10.1007/978-3-642-54568-9_3

Fan S, Wang Z, Xu W, Hou R, Meng D, Zhang M (2023) TensorFHE: achieving practical computation on encrypted data using GPGPU. In: 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 922–934. https://doi.org/10.1109/HPCA56546.2023.10071017

Gentry C, Sahai A, Waters B (2013) Homomorphic encryption from learning with errors: conceptually-simpler, asymptotically-faster, attribute-based. In: Advances in Cryptology–CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I, pp. 75–92 https://doi.org/10.1007/978-3-642-40041-4_5

Gupta S, Rosing TŠ (2021) Invited: Accelerating fully homomorphic encryption with processing in memory. In: 2021 58th ACM/IEEE Design Automation Conference (DAC). DAC, pp. 1335–1338. https://doi.org/10.1109/DAC18074.2021.9586285

Gupta S, Cammarota R, Šimunić T (2024) Memfhe: End-to-end computing with fully homomorphic encryption in memory. ACM Trans. Embed. Comput. Syst. 23(2) https://doi.org/10.1145/3569955

Han K, Hong S, Cheon JH, Park D (2019) Logistic regression on homomorphic encrypted data at scale. Proc. AAAI Conf. Artif. Intell. 33(01):9466–9471. https://doi.org/10.1609/aaai.v33i01.33019466

Han K, Ki D (2020) Better bootstrapping for approximate homomorphic encryption. In: Cryptographers'Track at the RSA Conference, pp. 364–390.https://doi.org/10.1007/978-3-030-40186-3_16

Jung W, Kim S, Ahn JH, Cheon JH, Lee Y (2021) Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. In: IACR Transactions on Cryptographic Hardware and Embedded Systems, 114–148 https://doi.org/10.46586/tches.v2021.i4.114-148

Kim A, Papadimitriou A, Polyakov Y (2022) Approximate homomorphic encryption with reduced approximation error. In: Cryptographers'Track at the RSA Conference. CT-RSA, pp. 120–144.https://doi.org/10.1007/978-3-030-95312-6_6

Kim J, Kim S, Choi J, Park J, Kim D, Ahn JH (2023) SHARP: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption. In: Proceedings of the 50th Annual International Symposium on Computer Architecture. ISCA '23, pp. 1–15, New York, NY, USA. https://doi.org/10.1145/3579371.3589053

Kim J, Lee G, Kim S, Sohn G, Rhu M, Kim J, Ahn JH (2022) Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse. In: 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO), pp. 1237–1254. https://doi.org/10.1109/micro56248.2022.00086

Kim S, Kim J, Kim MJ, Jung W, Kim J, Rhu M, Ahn JH (2022) BTS: an accelerator for bootstrappable fully homomorphic encryption. In: Proceedings of the 49th Annual International Symposium on Computer Architecture. ISCA '22, pp. 711–725, New York, NY, USA. https://doi.org/10.1145/3470496.3527415

Krizhevsky A, Hinton G et al (2009) Learning multiple layers of features from tiny images. Images. https://doi.org/10.1007/3-540-47977-5_32

LeCun Y, Cortes C, Burges C (2010) Mnist handwritten digit database. ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist **2**[SPACE]https://doi.org/10.1142/9789812772381_0022

Lee E, Lee J-W, Lee J, Kim Y-S, Kim Y, No J-S, Choi W (2022) Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In: Proceedings of the 39th International Conference on Machine Learning. ICML'2022, pp. 12403–12422.https://doi.org/10.1109/tdsc.2024.3448406

Liu F, Zhao W, Chen Y, Wang Z, He Z, Yang R, Tang Q, Yang T, Zhuo C, Jiang L (2022) PIM-DH: ReRAM-based processing-in-memory architecture for deep hashing acceleration. In: Proceedings of the 59th ACM/IEEE Design Automation Conference. DAC '22, pp. 1087–1092, New York, NY, USA. https://doi.org/10.1145/3489517.3530575

Montgomery PL (1985) Modular multiplication without trial division. Math. Computat. 44(170):519–521. https://doi.org/10.1090/s0025-5718-1985-0777282-x

Mouchet CV, Bossuat J-P, Troncoso-Pastoriza JR, Hubaux J-P (2020) Lattigo: a multiparty homomorphic encryption library in go. In: Proceedings of

the 8th Workshop on Encrypted Computing and Applied Homomorphic Cryptography, pp. 64–70 . https://doi.org/10.1109/icicis46948.2019.9014698

Muralimanohar N, Balasubramonian R, Jouppi NP (2009) Cacti 6.0: a tool to model large caches. HP laboratories 27, 28 https://doi.org/10.1109/mm.2008.2

Naresh Boddeti V (2018) Secure face matching using fully homomorphic encryption. In: 2018 IEEE 9th International Conference on Biometrics Theory, Applications and Systems (BTAS), pp. 1–10. https://doi.org/10.1109/BTAS.2018.8698601

Nejatollahi H, Gupta S, Imani M, Rosing TS, Cammarota R, Dutt N (2020) CryptoPIM: In-memory acceleration for lattice-based cryptographic hardware. In: 2020 57th ACM/IEEE Design Automation Conference (DAC). DAC '20, pp. 1–6. https://doi.org/10.1109/DAC18072.2020.9218730

Park Y, Wang Z, Yoo S, Lu WD (2022) Rm-ntt: an rram-based compute-in-memory number theoretic transform accelerator. IEEE J. Explorat. Solid-State Computat. Dev. Circ. 8(2):93–101. https://doi.org/10.1109/jxcdc.2022.3202517

Park J, Lee S, Lee J (2023) Ntt-pim: Row-centric architecture and mapping for efficient number-theoretic transform on pim. In: 2023 60th ACM/IEEE Design Automation Conference (DAC). DAC, pp. 1–6. https://doi.org/10.1109/DAC56929.2023.10247747

Perotti M, Cavalcante M, Wistoff N, Andri R, Cavigelli L, Benini L (2022) A "New Ara" for Vector Computing: An Open Source Highly Efficient RISC-V V 1.0 Vector Processor Design. In: 2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP), pp. 43–51. https://doi.org/10.1109/ASAP54787.2022.00017

Pollard JM (1971) The fast fourier transform in a finite field. Math. Computat. 25(114):365–374. https://doi.org/10.2307/2004932

RISC-V: RISC-V "V" Vector Extension. https://opensecura.googlesource.com/docs/+/3e75910d90b097ba2ac3a5113752c2c633d828b6/RiscVVectorSpecDoc.md (2023)

Samardzic N, Feldmann A, Krastev A, Devadas S, Dreslinski R, Peikert C, Sanchez D (2021) F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption. In: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO '21, pp. 238–252, New York, NY, USA. https://doi.org/10.1145/3466752.3480070

Samardzic N, Feldmann A, Krastev A, Manohar N, Genise N, Devadas S, Eldefrawy K, Peikert C, Sanchez D (2022) CraterLake: a hardware accelerator for efficient unbounded computation on encrypted data. In: Proceedings of the 49th Annual International Symposium on Computer Architecture. ISCA '22, pp. 173–187, New York, NY, USA. https://doi.org/10.1145/3470496.3527393

Shafiee A, Nag A, Muralimanohar N, Balasubramonian R, Strachan JP, Hu M, Williams RS, Srikumar V (2016) ISAAC: a convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In: 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA), pp. 14–26. https://doi.org/10.1109/ISCA.2016.12

Shivdikar K, Bao Y, Agrawal R, Shen M, Jonatan G, Mora E, Ingare A, Livesay N, AbellÁN JL, Kim J, Joshi A, Kaeli D (2023) GME: GPU-based microarchitectural extensions to accelerate homomorphic encryption. In: Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO '23, pp. 670–684, New York, NY, USA. https://doi.org/10.1145/3613424.3614279

Turan F, Roy SS, Verbauwhede I (2020) Heaws: An accelerator for homomorphic encryption on the amazon aws fpga. IEEE Trans. Comput. 69(8):1185–1196. https://doi.org/10.1109/TC.2020.2988765

Yang Y, Zhang H, Fan S, Lu H, Zhang M, Li X (2023) Poseidon: Practical Homomorphic Encryption Accelerator. In: 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 870–881. https://doi.org/10.1109/HPCA56546.2023.10070984

Zaruba F, Benini L (2019) The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 27(11), 2629–2640 https://doi.org/10.1109/TVLSI.2019.2926114

Zhang J, Imani M, Sadredini E (2023) Bp-ntt: Fast and compact in-sram number theoretic transform with bit-parallel modular multiplication. In: 2023 60th ACM/IEEE Design Automation Conference (DAC). DAC, pp. 1–6. https://doi.org/10.1109/DAC56929.2023.10247691

Zhu Y, Wang X, Ju L, Guo S (2023) Fxhenn: Fpga-based acceleration framework for homomorphic encrypted cnn inference. In: 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 896–907.https://doi.org/10.1109/hpca56546.2023.10071133

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.